A project report on

AI-POWERED MULTIMODAL KNOWLEDGE RETRIEVAL AND DECISION SUPPORT SYSTEM

Submitted in partial fulfilment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

SYAMAKURI HEMA HARSHITH (22BCE20290)

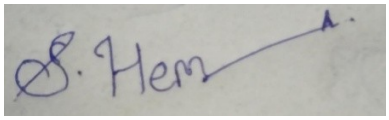
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SCOPE)

May, 2026

DECLARATION

I hereby declare that the thesis entitled "**AI-POWERED MULTIMODAL KNOWLEDGE RETRIEVAL AND DECISION SUPPORT SYSTEM**" submitted by Syamakuri Hema Harshith (22BCE20290) for the award of the degree of Bachelor of Technology in Computer Science and Engineering-core, VIT is a record of Bonafide work carried out by me under the supervision of Dr. Surendra Reddy Vinta.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.



Place: Amaravati

Date: 16-05-2026

Signature of the Candidate

CERTIFICATE

This is to certify that the Internship titled **"AI-Powered Multimodal Knowledge Retrieval and Decision Support System"** that is being submitted by SYAMAKURI HEMA HARSHITH (22BCE20290) is in partial fulfillment of the requirements for the award of Bachelor of Technology, is a record of bonafide work done under my guidance. The contents of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for award of any degree or diploma and the same is certified.

TRIAD AI PRIVATE LIMITED

Name of the Project Guide/Team Lead/Project Manager
(with company seal)

The thesis is satisfactory / unsatisfactory

Internal Examiner **External Examiner**

Approved by

DEAN

School of Computer Science and Engineering

3

Triad AI Pvt Ltd.

CIN:U62020TS2024PTC189592
GST:36AALCT2873J1ZT

Date: 12-05-2026

INTERNSHIP COMPLETION LETTER

TO WHOMSOEVER IT MAY CONCERN

This is to certify that Mr. Hema Harshith Syamakuri, a student of VIT-AP University, was associated with our organization, Triad AI Pvt. Ltd., Hyderabad, in the capacity of Intern for the period 10th January 2026 to 12th May 2026.

He has successfully completed the Internship Program to our satisfaction and was actively involved in the Indian Coast Guard Project, contributing to tasks related to AI-driven systems and logistics optimization.

We appreciate his sincere efforts and dedication during the internship period, and we wish him all the best in his future endeavors.

For Triad AI Pvt.Ltd.

Dr. T RajyaLakshmi
Head of Business Development
Triad AI Pvt. Ltd.

ABSTRACT

The Materiel Directorate of the Indian Coast Guard (ICG) is responsible for managing and maintaining a large and continuously expanding repository of technical and operational documents related to procurement, inspection, maintenance, compliance, and engineering support activities. These documents include equipment maintenance manuals, procurement specifications, vendor quotations, technical drawings, engineering schematics, inspection reports, policy circulars, Statement of Technical Requirements (SOTR) documents, calibration records, and maintenance logs. Over the years, these records have been distributed across multiple disconnected repositories such as departmental drives, local file systems, email archives, scanned document collections, and physical binders. Due to the absence of a centralized intelligent retrieval system, officers are often required to manually search through large volumes of heterogeneous documents in order to identify relevant technical specifications, procurement standards, inspection findings, and maintenance references. This manual workflow not only consumes significant operational time but also results in inefficiencies, duplication of effort, and dependency on the experience and institutional knowledge of senior personnel.

One of the major challenges identified within the existing system is the inability to perform cross-document reasoning across different categories of records. In practical scenarios, officers may need to verify whether a particular engineering drawing complies with an existing procurement specification, determine whether inspection findings violate maintenance tolerances specified in technical manuals, or prepare a Statement of Technical Requirements (SOTR) by consolidating information from multiple independent documents. Conventional keyword-based search systems are insufficient for such tasks because they lack semantic understanding and cannot correlate information distributed across different document types. Furthermore, engineering drawings and scanned schematics stored in formats such as PDF, TIFF, PNG, or JPEG cannot be effectively searched using traditional search mechanisms because most of the critical information exists in visual annotations, dimensional labels, title blocks, and diagrammatic structures rather than machine-readable text. As a result, valuable operational and technical knowledge remains inaccessible, and institutional expertise is frequently lost when experienced officers are transferred or retired.

To address these challenges, this project proposes the development of an AI-powered Multimodal Knowledge Retrieval and Decision Support System based on Retrieval-Augmented Generation (RAG) architecture. The proposed system is designed as a fully on-premise and air-gapped platform to ensure that sensitive defense-related data never leaves the secure infrastructure of the Indian Coast Guard. The system integrates technologies such as natural language processing, semantic vector retrieval, OCR-based text extraction, multimodal document understanding, and large language models (LLMs) to create a unified intelligent knowledge management platform. Documents are parsed, segmented into contextual chunks, converted into dense vector embeddings, and stored within the Qdrant vector database for efficient semantic retrieval. The system supports multiple file formats including digital PDFs, scanned documents, Word files,

Excel sheets, and engineering drawings. Scanned documents are processed using OCR techniques, while engineering drawings are indexed through a multimodal three-path pipeline consisting of OCR-based annotation extraction, Visual Question Answering (VQA), and CLIP-based visual embeddings. This enables drawings to become searchable through textual descriptions, semantic similarity, and visual matching.

The query-processing pipeline allows officers to interact with the system using natural language queries without requiring technical database expertise. Retrieval is performed using hybrid mechanisms including semantic vector search, BM25 keyword retrieval, OCR field matching, and visual similarity retrieval. Retrieved passages and metadata are combined using Reciprocal Rank Fusion (RRF) and passed to a locally hosted large language model for grounded answer generation. The language model is configured with strict grounding constraints to ensure factual consistency and citation-based responses. In addition to intelligent query answering, the system also supports automated generation of Statement of Technical Requirements (SOTR) documents by synthesizing information from procurement specifications, maintenance manuals, inspection reports, and engineering drawings. Each generated response includes citations to the original source documents, enabling officers to verify the authenticity of retrieved information.

The entire system is deployed using Docker-based containerized infrastructure and operates completely within the Indian Coast Guard network to satisfy air-gap security requirements. All AI models, embedding systems, OCR engines, and retrieval components are hosted locally without dependency on external cloud services. The proposed solution significantly improves operational efficiency by reducing document retrieval time, enabling intelligent cross-document reasoning, automating repetitive technical documentation tasks, and preserving institutional knowledge within the organization. By integrating multimodal retrieval, semantic search, grounded large language model reasoning, and secure on-premise deployment, this project presents a scalable and practical AI-driven solution for defense-oriented technical knowledge management and decision support systems.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr. Surendra Reddy Vinta , Assistant Professor Senior Grade 2, SCOPE, VIT-AP, for his constant guidance, continual encouragement, understanding; more than all, he taught me patience in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of Artificial Intellegence.

I would like to express my gratitude to Dr G. Viswanathan, Dr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. P. Arulmozhivarman, and Dr. S Sudhakar Ilango, SCOPE, for providing with an environment to work in and for his inspiration during the tenure of the course.

In jubilant mood I express ingeniously my whole-hearted thanks to Dr. S Sudhakar Ilango. Professor Grade 1, all teaching staff and members working as limbs of our university for their not-self-centered enthusiasm coupled with timely encouragements showered on me with zeal, which prompted the acquirement of the requisite knowledge to finalize my course study successfully. I would like to thank my parents for their support.

It is indeed a pleasure to thank my friends who persuaded and encouraged me to take up and complete this task. At last but not least, I express my gratitude and appreciation to all those who have helped me directly or indirectly toward the successful completion of this project.

Place: Amaravati — Syamakuri Hema Harshith

Date: 16-05-2026 — Name of the student

CONTENTS Page No.

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS

AI	Artificial Intelligence
API	Application Programming Interface
BM25	Best Matching 25
CLIP	Contrastive Language–Image Pre-Training
CPU	Central Processing Unit
CRR	Cash Reserve Ratio
CSS	Cascading Style Sheets
DI	Data Interpretation
DOC	Microsoft Word Document
DOCX	Microsoft Word Open XML Document
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
ICG	Indian Coast Guard
IMPS	Immediate Payment Service
LLM	Large Language Model
MPC	Monetary Policy Committee
NBFC	Non-Banking Financial Company
NEFT	National Electronic Funds Transfer
NLP	Natural Language Processing
NPA	Non-Performing Asset
OCR	Optical Character Recognition
PCA	Prompt Corrective Action
PDF	Portable Document Format

PE	Polyethylene
PHA	Polyhydroxyalkanoates
PNG	Portable Network Graphics
PS	Polystyrene
Q&A	Question and Answer
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
RRF	Reciprocal Rank Fusion
RTGS	Real Time Gross Settlement
SI/CI	Simple Interest / Compound Interest
SLR	Statutory Liquidity Ratio
SOTR	Statement of Technical Requirements
SSD	Solid State Drive
TF-IDF	Term Frequency–Inverse Document Frequency
TIFF	Tagged Image File Format
UI	User Interface
UPI	Unified Payments Interface
VQA	Visual Question Answering
XLS	Microsoft Excel Spreadsheet
XLSX	Microsoft Excel Open XML Spreadsheet
VRAM	Video Random Access Memory

Chapter 1

Introduction

The Materiel Directorate of the Indian Coast Guard (ICG) manages a vast collection of technical and operational documents that are essential for procurement, inspection,

maintenance, compliance verification, and engineering support activities. These documents include procurement specifications, equipment maintenance manuals, technical reports, inspection records, calibration data, vendor quotations, engineering drawings, policy circulars, maintenance logs, and Statement of Technical Requirements (SOTR) documents. Over time, the volume of these documents has increased significantly, creating major challenges in information organization, accessibility, and knowledge management within the organization. Most of these records are distributed across fragmented repositories such as departmental drives, local file systems, scanned PDF archives, email repositories, and physical binders, with no unified retrieval mechanism available for officers to efficiently access required information.

In the current workflow, officers are often required to manually search through multiple documents and repositories to locate relevant procurement standards, technical parameters, inspection findings, maintenance procedures, or engineering references. This process is highly time-consuming and inefficient, especially when the required information is distributed across different document categories. For example, verifying whether an engineering drawing complies with an existing procurement specification may require manual comparison between drawings, inspection reports, and maintenance manuals. Similarly, generating a Statement of Technical Requirements (SOTR) requires officers to gather and cross-reference information from multiple disconnected sources, which may take several hours or even days. The absence of a centralized intelligent retrieval mechanism also results in duplication of work and dependency on the institutional knowledge of experienced personnel. When officers are transferred or retired, valuable knowledge regarding document locations, procurement history, and technical references is often lost permanently.

Another significant challenge arises from the presence of engineering drawings and technical schematics within the document corpus. A large number of engineering drawings are stored in scanned PDF, TIFF, PNG, or JPEG formats containing dimensional annotations, assembly notes, tolerance values, and title block information. Traditional keyword-based search systems are incapable of effectively processing such visual information because the content is not directly machine-readable. As a result, officers cannot search drawings using natural language descriptions or visual similarity. Existing systems can retrieve drawings only when the exact drawing number or filename is known in advance. This limitation severely affects technical verification workflows and reduces operational efficiency within the Materiel Directorate.

To address these challenges, this project proposes the development of an AI-powered Multimodal Knowledge Retrieval and Decision Support System based on Retrieval-Augmented Generation (RAG) architecture. The proposed system is designed to function as a centralized intelligent knowledge management platform capable of processing, indexing, retrieving, and reasoning across multiple categories of technical documents and engineering drawings. Unlike conventional AI systems that rely solely on pre-trained model knowledge, the RAG-based approach dynamically retrieves relevant information from indexed documents before generating responses. This retrieval-first approach improves factual grounding, reduces hallucination, and ensures that generated outputs remain directly linked to source documents.

The proposed system operates entirely within the Indian Coast Guard infrastructure as a fully on-premise and air-gapped deployment. This design is essential because

defense-related technical documents contain sensitive operational information that cannot be transmitted to external cloud services or third-party AI APIs. All components of the system, including large language models, embedding models, OCR engines, vector databases, and multimodal reasoning systems, are hosted locally within secure GPU-enabled servers. Docker-based containerized deployment is used to simplify service orchestration, scalability, and infrastructure management while maintaining strict network isolation policies.

The document ingestion pipeline supports multiple file formats including digital PDFs, scanned documents, Word files, Excel spreadsheets, and engineering drawings. Digital documents are processed using specialized parsing libraries, while scanned documents undergo OCR-based text extraction. After extraction, the textual content is divided into contextual chunks and converted into dense vector embeddings using transformer-based embedding models. These embeddings are then stored inside the Qdrant vector database, which enables high-speed semantic similarity retrieval. In addition to semantic vector retrieval, the system also incorporates BM25 keyword-based retrieval to improve exact-match search performance for part numbers, specification IDs, and technical codes. Hybrid retrieval is further enhanced using Reciprocal Rank Fusion (RRF), which combines results from multiple retrieval streams to improve overall relevance and retrieval quality.

One of the key innovations of the proposed system is the multimodal engineering drawing retrieval pipeline. Each engineering drawing is processed through three parallel indexing paths. The first path uses OCR extraction to identify dimensions, tolerance values, annotations, material specifications, and title block details from the drawing. The second path uses Visual Question Answering (VQA) models to generate semantic descriptions and reasoning-oriented representations of the engineering schematics. The third path generates CLIP-based visual embeddings that enable image similarity search across visually related drawings. Together, these three indexing methods transform each drawing into multiple independently searchable representations, allowing officers to retrieve engineering data through text queries, semantic matching, or visual similarity search.

The query-processing pipeline allows officers to interact with the system using natural language queries in plain English without requiring technical database expertise or structured query languages. When a query is submitted, the system first performs query understanding and embedding generation. Multiple retrieval mechanisms are then executed in parallel, including semantic vector search, BM25 keyword search, OCR-based annotation retrieval, and CLIP visual similarity search for drawing-related queries. The language model is constrained through strict grounding instructions to ensure that every generated response is supported by retrieved evidence and accompanied by proper citations. If sufficient information is unavailable, the model is instructed to refuse unsupported claims rather than generating hallucinated responses.

In addition to intelligent document retrieval and question answering, the system also supports automated generation of Statement of Technical Requirements (SOTR) documents. The SOTR module retrieves relevant information from procurement specifications, maintenance manuals, inspection reports, vendor records, and engineering drawings to automatically generate structured draft documents with supporting citations. This capability significantly reduces the manual effort involved in

preparing procurement and compliance documentation while maintaining traceability and verification support.

The proposed system is expected to significantly improve operational efficiency within the Indian Coast Guard by reducing document retrieval time, enabling intelligent cross-document reasoning, automating repetitive documentation workflows, and preserving institutional technical knowledge. Officers will be able to retrieve accurate technical information within seconds instead of manually searching through large repositories for extended periods. By integrating semantic retrieval, multimodal document understanding, grounded large language model reasoning, and secure on-premise deployment, this project demonstrates a scalable and practical AI-driven solution for defense-oriented knowledge management and decision support systems.

1.1 BACKGROUND

The rapid digitization of technical and operational processes within defense and maritime organizations has resulted in the generation of massive volumes of structured and unstructured data. Organizations such as the Indian Coast Guard rely heavily on technical documentation for procurement management, equipment maintenance, inspection procedures, engineering support, compliance verification, and operational decision-making. These documents form the foundation for critical engineering and administrative activities and include procurement specifications, maintenance manuals, inspection reports, vendor communications, engineering drawings, calibration records, policy circulars, and Statement of Technical Requirements (SOTR) documents. As the volume of such records continues to grow over time, managing and retrieving information efficiently becomes increasingly difficult, particularly when documents are distributed across multiple isolated repositories without a centralized knowledge management framework.

Traditionally, document retrieval within technical organizations has relied on folder-based storage systems and keyword-based search methods. Although such approaches are adequate for small and well-structured repositories, they become inefficient when dealing with heterogeneous technical documents spread across multiple departments and formats. In many cases, officers must manually navigate through large collections of PDFs, scanned files, spreadsheets, and archived documents to locate relevant information. This process consumes substantial operational time and often depends on the personal experience and institutional knowledge of senior personnel familiar with the document ecosystem. As a result, the efficiency of information retrieval is heavily dependent on individual expertise rather than the capabilities of the information system itself. When experienced personnel are transferred or retire, critical knowledge regarding procurement history, maintenance references, or technical specifications may become inaccessible to newer officers.

A major limitation of conventional search systems is their inability to perform semantic understanding and cross-document reasoning. Traditional keyword-based search engines operate primarily by matching exact words or phrases present within documents. However, technical and engineering domains often contain semantically related concepts expressed using different terminology. For example, a maintenance report may refer to “bearing play” while a procurement specification may describe the same concept as “bearing clearance tolerance.” Conventional search systems are unable

to recognize such semantic relationships effectively, resulting in incomplete retrieval results. Furthermore, officers frequently require information synthesis across multiple document categories rather than retrieval from a single source. Tasks such as verifying whether a technical drawing complies with procurement standards or identifying recurring inspection failures across multiple reports require reasoning across disconnected datasets. Existing retrieval systems lack the capability to correlate information distributed across manuals, inspection records, procurement documents, and engineering drawings simultaneously.

The challenge becomes even more complex in the case of engineering drawings and technical schematics. A significant portion of technical information within defense organizations exists in graphical form rather than machine-readable text. Engineering drawings contain dimensional annotations, tolerances, assembly structures, material specifications, title block information, and visual relationships between components. Most of these drawings are stored in scanned PDF, TIFF, PNG, or JPEG formats that cannot be effectively processed using standard text retrieval systems. Conventional OCR-based extraction techniques alone are often insufficient because technical drawings involve spatial relationships, symbols, labels, and engineering notations that require contextual interpretation. Consequently, drawings can generally be retrieved only when the exact drawing number or file identifier is already known. This limitation significantly reduces the usability of engineering repositories and affects technical verification workflows within the organization.

Recent advancements in Artificial Intelligence, Natural Language Processing (NLP), and Large Language Models (LLMs) have introduced new possibilities for intelligent knowledge management systems. Modern transformer-based language models are capable of understanding contextual meaning, performing semantic similarity matching, generating human-like responses, and reasoning across multiple information sources. Retrieval-Augmented Generation (RAG) has emerged as an effective architecture for combining semantic retrieval with grounded language model reasoning. In a RAG framework, relevant information is first retrieved from external knowledge repositories and then supplied to the language model for answer generation. This approach significantly improves factual grounding, reduces hallucination, and allows generated responses to remain directly traceable to source documents.

In parallel with advancements in text-based AI systems, multimodal AI models have enabled the processing and understanding of visual content such as images, diagrams, and engineering drawings. Models such as CLIP and multimodal Visual Question Answering (VQA) systems are capable of generating semantic representations of images and correlating them with textual descriptions. These technologies make it possible to transform engineering drawings into searchable semantic knowledge units rather than static visual files. By combining OCR extraction, visual embeddings, and multimodal reasoning, engineering schematics can be indexed and retrieved through natural language queries and visual similarity search mechanisms.

The Indian Coast Guard environment presents a particularly suitable use case for such intelligent retrieval systems because of the diversity, scale, and sensitivity of its technical document repositories. The Materiel Directorate handles procurement

workflows, maintenance planning, inspection activities, and compliance verification involving multiple document categories that must often be analyzed together. In addition, defense organizations require strict security controls and air-gapped infrastructure due to the sensitive nature of operational and technical information. Cloud-based AI systems are therefore unsuitable for deployment in such environments because sensitive documents cannot be transmitted to external servers or third-party APIs. This necessitates the development of a fully on-premise AI-driven retrieval architecture capable of operating securely within isolated internal networks.

The proposed project addresses these challenges through the development of a Multimodal Retrieval-Augmented Generation system specifically designed for defense-oriented technical knowledge management. The system combines semantic vector retrieval, OCR-based document processing, multimodal engineering drawing understanding, grounded large language model reasoning, and automated SOTR generation within a unified secure platform. By enabling natural language interaction with technical repositories, the proposed solution aims to reduce retrieval time, improve operational efficiency, automate repetitive documentation workflows, and preserve institutional knowledge within the Indian Coast Guard. The integration of multimodal retrieval and grounded AI reasoning represents a significant advancement over traditional document management systems and demonstrates the growing role of artificial intelligence in secure enterprise-scale knowledge management applications.

1.2 PROBLEM STATEMENT

The Materiel Directorate of the Indian Coast Guard (ICG) manages a large and continuously growing collection of technical and operational documents related to procurement, inspection, maintenance, engineering support, and compliance verification. These documents include procurement specifications, maintenance manuals, engineering drawings, inspection reports, vendor communications, policy circulars, calibration records, and Statement of Technical Requirements (SOTR) documents. Over time, these records have become distributed across multiple disconnected repositories such as departmental drives, local file systems, scanned archives, email repositories, and physical binders, resulting in the absence of a unified and intelligent document retrieval mechanism.

The existing document management approach relies heavily on manual searching and traditional keyword-based retrieval methods, which are inefficient for handling large volumes of heterogeneous technical data. Officers are required to spend significant time manually locating relevant information from multiple independent sources in order to verify technical specifications, analyze inspection findings, compare procurement standards, or prepare SOTR documents. This process is time-consuming, repetitive, and highly dependent on the institutional knowledge and prior experience of individual personnel. As a result, operational efficiency is reduced, decision-making workflows are delayed, and valuable technical knowledge may be lost when experienced officers are transferred or retired.

A major challenge within the existing system is the inability to perform semantic and cross-document reasoning across different categories of records. Conventional search

systems are capable of retrieving documents only through exact keyword matching and cannot understand contextual relationships between technical concepts expressed using different terminology. Furthermore, existing systems cannot correlate information distributed across procurement specifications, maintenance manuals, inspection reports, and engineering drawings simultaneously. Tasks such as verifying whether an engineering drawing complies with procurement standards or identifying recurring equipment deficiencies across inspection reports require cross-modal and cross-document analysis that current retrieval systems are unable to provide effectively.

Another significant limitation involves engineering drawings and technical schematics, which contain a large amount of critical information in visual form. Most drawings are stored as scanned PDF, TIFF, PNG, or JPEG files containing dimensional annotations, tolerance values, title blocks, material specifications, and assembly notes that are not searchable using traditional text-based systems. Existing retrieval methods generally require officers to know the exact drawing number or filename beforehand, making natural language-based drawing retrieval practically impossible.

In addition to retrieval challenges, the preparation of Statement of Technical Requirements (SOTR) documents remains largely manual. Officers must collect and consolidate information from multiple procurement specifications, maintenance manuals, engineering drawings, and inspection reports to prepare structured technical requirement documents. This manual workflow consumes considerable time and increases the possibility of missing relevant references, inconsistencies, or outdated specifications.

Therefore, there is a need for an intelligent, secure, and scalable knowledge management system capable of processing heterogeneous technical documents, understanding semantic relationships, performing cross-document reasoning, retrieving engineering drawings through multimodal search, and generating grounded responses with source citations. The system must operate entirely within the Indian Coast Guard infrastructure without dependency on external cloud services due to the sensitive nature of defense-related technical information. The proposed AI-powered Multimodal Retrieval-Augmented Generation (RAG) system addresses these challenges by integrating semantic retrieval, multimodal document understanding, vector databases, large language models, and automated SOTR generation within a fully on-premise and air-gapped environment.

1.3 ORGANIZATION OF THE PROJECT

The remaining chapters of this report are organized as follows:

• **Chapter 2** presents the literature survey, discusses existing document retrieval and knowledge management approaches, identifies limitations in traditional search

systems, and explains the objectives and significance of the proposed Retrieval-Augmented Generation (RAG) based framework.

• **Chapter 3** describes the methodology and system architecture used in the project, including document ingestion, OCR processing, vector embedding generation, multimodal engineering drawing retrieval, hybrid search mechanisms, large language model integration, and automated SOTR generation workflow.

• **Chapter 4** discusses the implementation and results of the proposed system, including deployment architecture, retrieval performance, query-response generation, multimodal search outputs, system evaluation, hardware and software requirements, and limitations encountered during development.

• **Chapter 5** presents the conclusion of the project and outlines possible future enhancements such as improved multimodal reasoning, advanced reranking techniques, multilingual support, and real-time document synchronization capabilities.

• The **references** section includes all research papers, technical resources, documentation, and external sources referred to during the development of the project.

• The **appendix** contains supplementary material related to the project, including sample outputs, architecture diagrams, workflow illustrations, configuration details, and additional implementation information.

1.4 SCOPE OF THE PROJECT

The scope of this project involves the design and development of an AI-powered Multimodal Knowledge Retrieval and Decision Support System for the Materiel Directorate of the Indian Coast Guard (ICG). The system is intended to address the growing challenges associated with managing, retrieving, and utilizing large volumes of technical and operational documents distributed across multiple repositories. The proposed framework focuses on enabling intelligent semantic retrieval, multimodal document understanding, grounded natural language query answering, and automated technical document generation within a secure and fully on-premise environment.

The project covers the processing and indexing of multiple categories of technical documents including procurement specifications, equipment maintenance manuals, inspection reports, policy circulars, calibration records, vendor communications, engineering drawings, and Statement of Technical Requirements (SOTR) documents. The system supports heterogeneous file formats such as digital PDFs, scanned PDFs, Word documents, spreadsheets, and image-based engineering schematics. The scope of the project includes the development of a document ingestion pipeline capable of extracting textual and visual information from these documents using OCR-based extraction, semantic chunking, and multimodal processing techniques.

A major scope of the project lies in the implementation of semantic retrieval using Retrieval-Augmented Generation (RAG) architecture. The proposed system enables

officers to interact with the document repository using natural language queries rather than relying on manual searching or exact keyword matching. By integrating transformer-based embedding models and vector databases, the system performs semantic similarity search across large document collections and retrieves contextually relevant information from multiple sources simultaneously. The retrieval framework also incorporates hybrid retrieval mechanisms combining semantic vector search and BM25 keyword retrieval to improve search accuracy for technical terminology, specification IDs, and equipment references.

The project further extends to multimodal retrieval and analysis of engineering drawings and technical schematics. Since conventional text-based retrieval systems cannot effectively process graphical engineering information, the proposed system incorporates OCR extraction, Visual Question Answering (VQA), and CLIP-based visual embeddings to convert engineering drawings into searchable semantic representations. This enables officers to retrieve drawings using textual descriptions, technical annotations, and visual similarity search methods.

Another important scope of the project includes the integration of locally hosted Large Language Models (LLMs) for grounded answer generation and technical reasoning. The system generates responses only from retrieved evidence and provides source citations to ensure traceability and factual consistency. The project also includes an automated Statement of Technical Requirements (SOTR) generation module capable of synthesizing information from procurement specifications, inspection reports, maintenance manuals, and engineering drawings into structured draft documents. This significantly reduces the manual effort required for procurement documentation workflows.

The proposed system is designed specifically for secure defense-oriented deployment environments. Therefore, the project scope includes fully on-premise and air-gapped deployment using Docker-based containerized infrastructure without dependency on external cloud services or third-party APIs. All AI models, vector databases, OCR engines, and retrieval components are hosted locally within the organization's infrastructure to ensure confidentiality and compliance with defense security requirements. The overall architecture and methodology can be extended to other defense organizations, engineering departments, public sector enterprises, and large-scale technical institutions that manage extensive document repositories. Future expansion of the system may include multilingual document support, advanced reranking mechanisms, predictive maintenance analytics, role-based personalization, and real-time synchronization with enterprise document management systems. By integrating semantic retrieval, multimodal understanding, grounded AI reasoning, and secure deployment architecture, the project establishes a scalable and practical framework for intelligent technical knowledge management and decision support systems.

Chapter 2

2.1 LITERATURE SURVEY

The rapid increase in digital technical documentation across defense, engineering, and industrial organizations has created a growing demand for intelligent information

retrieval and knowledge management systems. Traditional document management systems primarily rely on folder-based organization and keyword-driven search mechanisms for accessing stored information. Although such systems are effective for small and structured datasets, they become increasingly inefficient when dealing with large volumes of heterogeneous technical documents distributed across multiple repositories. Several studies have identified that conventional search systems fail to support semantic understanding, contextual reasoning, and cross-document correlation, especially in environments where technical information is scattered across manuals, inspection records, procurement specifications, engineering drawings, and maintenance reports.

Early document retrieval systems were primarily based on lexical search techniques such as TF-IDF and BM25 ranking algorithms. These systems retrieved documents based on exact keyword matching and frequency analysis. BM25, in particular, became widely adopted in enterprise search systems because of its ability to rank documents according to term relevance and occurrence probability. However, lexical retrieval systems suffer from major limitations in technical domains because semantically related concepts may be expressed using different terminology. For example, technical manuals and inspection reports may describe identical engineering conditions using different words, making exact keyword retrieval insufficient for reliable information discovery. Researchers observed that keyword-based retrieval systems often fail to identify contextual similarity between related technical descriptions, thereby reducing retrieval accuracy in engineering and maintenance applications.

Advancements in Natural Language Processing (NLP) and transformer-based language models significantly improved semantic understanding capabilities in document retrieval systems. Transformer architectures such as BERT introduced contextual embeddings capable of representing semantic meaning rather than simple keyword frequency. Sentence-transformer models and dense embedding frameworks further enabled semantic similarity search by converting textual content into high-dimensional vector representations. Vector-based semantic retrieval systems demonstrated improved performance in identifying contextually related documents even when exact keywords were absent. The emergence of vector databases such as FAISS, Pinecone, Milvus, and Qdrant provided scalable infrastructure for storing and retrieving dense vector embeddings efficiently across large enterprise datasets.

Although semantic retrieval systems improved contextual search capabilities, researchers identified a major limitation in standalone Large Language Models (LLMs). Conventional LLMs generate responses using knowledge learned during pretraining and fine-tuning processes, which may lead to hallucinations, outdated responses, and lack of traceability. In technical and defense-oriented applications, such unreliable outputs are unacceptable because engineering decisions require verifiable evidence and source references. To address these limitations, Retrieval-Augmented Generation (RAG) emerged as a hybrid architecture combining semantic retrieval with grounded language model reasoning. In RAG systems, relevant document passages are first retrieved from external knowledge repositories and then supplied to the language model during response generation. This approach significantly improves factual grounding, reduces hallucination, and allows generated outputs to remain directly traceable to source documents.

Several recent studies demonstrated the effectiveness of RAG-based systems in enterprise knowledge management, technical support automation, legal document retrieval, and research assistance platforms. Researchers found that hybrid retrieval methods combining dense vector search with lexical retrieval algorithms such as BM25 produced better retrieval accuracy than using either approach independently. Techniques such as Reciprocal Rank Fusion (RRF) were introduced to combine rankings from multiple retrieval streams, improving retrieval relevance across diverse document types. Hybrid retrieval became particularly useful in technical environments containing engineering identifiers, specification numbers, equipment labels, and domain-specific terminology where both semantic understanding and exact keyword matching are required simultaneously.

Another major area of research relevant to this project involves multimodal document understanding. Engineering organizations often store critical technical information within drawings, schematics, diagrams, and scanned blueprints rather than plain textual documents. Traditional OCR-based extraction systems can identify visible text but fail to capture structural and contextual information present within engineering diagrams. Recent advancements in multimodal AI models introduced systems capable of jointly understanding text and visual information. Models such as CLIP demonstrated the ability to map images and textual descriptions into shared embedding spaces, enabling image similarity search and cross-modal retrieval. Similarly, Visual Question Answering (VQA) systems and multimodal large language models such as LLaVA enabled semantic reasoning over engineering drawings and image-based documents.

Research in multimodal retrieval showed that combining OCR extraction, visual embeddings, and semantic reasoning significantly improves accessibility of engineering repositories. Studies conducted in industrial maintenance and manufacturing environments demonstrated that multimodal indexing allows technical drawings to become searchable through natural language descriptions, dimensional annotations, and visual similarity comparisons. These advancements are highly relevant for defense-oriented technical document systems where engineering schematics contain operationally critical information that cannot be effectively processed through conventional search mechanisms.

In parallel with retrieval advancements, secure deployment of AI systems became an important research focus in defense and enterprise environments. Many organizations handling sensitive technical information cannot rely on cloud-based AI services due to confidentiality and compliance requirements. As a result, researchers proposed fully on-premise AI architectures using locally hosted vector databases, embedding models, and large language models deployed within air-gapped infrastructure. Containerization technologies such as Docker further simplified deployment and orchestration of complex AI pipelines while maintaining isolation and infrastructure portability.

2.2 GAPS IDENTIFIED

• Existing document retrieval systems rely heavily on exact keyword matching and are unable to understand semantic relationships between technical terms and engineering concepts.

• Traditional search mechanisms cannot perform cross-document reasoning across procurement specifications, inspection reports, maintenance manuals, and engineering drawings simultaneously.

• Most enterprise retrieval systems are designed primarily for textual documents and provide very limited support for engineering drawings, schematics, and scanned technical diagrams.

• Conventional OCR-based systems can extract visible text from scanned documents but fail to understand contextual and structural information present in technical drawings.

• Existing systems generally require users to know exact filenames, document IDs, or drawing numbers beforehand, making natural language-based retrieval difficult.

• Technical knowledge remains distributed across fragmented repositories such as departmental drives, local systems, scanned archives, and physical records without a centralized intelligent knowledge management framework.

• Current retrieval workflows depend heavily on manual searching and institutional knowledge of experienced personnel, leading to inefficiency and knowledge loss during personnel transfer or retirement.

• Most conventional systems do not support multimodal retrieval using textual descriptions, OCR annotations, and visual similarity simultaneously.

• Existing AI-based retrieval systems often suffer from hallucination issues because generated responses are not always grounded in verified source documents.

• Many available Large Language Model (LLM) solutions depend on cloud-based APIs, making them unsuitable for defense and security-sensitive environments requiring air-gapped deployment.

• Existing enterprise search platforms generally lack citation-based response generation, making verification and traceability of retrieved information difficult.

• Current systems do not provide automated generation of technical documents such as Statement of Technical Requirements (SOTR) using information synthesized from multiple repositories.

• Most retrieval systems do not integrate semantic vector retrieval, BM25 keyword search, multimodal engineering understanding, and grounded language model reasoning within a single unified architecture.

• Existing document management approaches are not scalable enough to efficiently handle continuously growing repositories of heterogeneous technical and operational records.

• There is limited availability of AI-driven retrieval frameworks specifically designed for defense-oriented technical knowledge management and engineering support applications.

2.3 OBJECTIVES

• To develop an intelligent AI-powered knowledge retrieval and decision support system for the Materiel Directorate of the Indian Coast Guard.

• To create a centralized platform for managing and retrieving technical and operational documents distributed across multiple repositories.

• To implement a Retrieval-Augmented Generation (RAG) based framework for semantic and context-aware document retrieval.

• To enable natural language-based querying of technical documents without requiring manual file searching or exact keyword matching.

• To process and index multiple document formats including PDFs, scanned documents, Word files, spreadsheets, and engineering drawings.

• To extract textual information from scanned documents and image-based files using Optical Character Recognition (OCR) techniques.

• To generate semantic vector embeddings for document chunks and store them efficiently within a vector database.

• To implement hybrid retrieval using semantic vector search and BM25 keyword retrieval for improved search accuracy.

• To support multimodal retrieval of engineering drawings using OCR extraction, visual embeddings, and semantic descriptions.

• To integrate multimodal AI models for understanding and indexing technical schematics and engineering diagrams.

• To develop a grounded response generation mechanism using locally hosted Large Language Models (LLMs).

• To ensure that generated responses are supported by retrieved evidence and include proper source citations.

• To reduce hallucination and improve factual consistency in AI-generated responses through grounded retrieval mechanisms.

• To automate the generation of Statement of Technical Requirements (SOTR) documents using retrieved technical information from multiple sources.

• To reduce the manual effort and time required for procurement documentation and technical information retrieval.

• To preserve institutional technical knowledge and reduce dependency on individual personnel experience.

• To design the system as a fully on-premise and air-gapped deployment suitable for defense-oriented environments.

• To ensure secure handling of sensitive technical and operational data without dependency on external cloud services.

• To improve operational efficiency, technical decision-making, and knowledge accessibility within the Indian Coast Guard Materiel Directorate.

• To develop a scalable and extensible architecture that can be adapted for other defense and engineering organizations in the future.

2.4 APPLICATIONS OF THE SYSTEM

The proposed AI-powered Multimodal Knowledge Retrieval and Decision Support System has wide-ranging applications within the Indian Coast Guard Materiel Directorate and other defense-oriented technical environments. One of the primary applications of the system is intelligent technical document retrieval. Officers can use natural language queries to quickly access information from procurement specifications, maintenance manuals, inspection reports, policy circulars, vendor records, and engineering documents without manually searching through multiple repositories. This significantly reduces the time required to locate technical information and improves operational efficiency during maintenance, procurement, and compliance-related activities.

Another major application of the system is procurement support and technical specification verification. During procurement workflows, officers are often required to compare technical specifications, evaluate vendor submissions, verify compliance with standards, and reference historical procurement records. The proposed system enables semantic retrieval of related procurement documents and allows officers to identify relevant technical parameters, past inspection observations, and vendor-related information efficiently. This improves the accuracy and consistency of procurement decision-making while reducing manual effort.

The system also has important applications in automated Statement of Technical Requirements (SOTR) generation. Preparation of SOTR documents traditionally requires officers to collect and consolidate information from multiple independent technical sources. The proposed framework automates this process by retrieving relevant specifications, maintenance procedures, inspection findings, and engineering references from indexed repositories and generating structured draft SOTR documents with supporting citations. This reduces documentation time significantly and helps maintain consistency across procurement and technical documentation workflows.

A significant application of the system lies in engineering drawing and schematic retrieval. Traditional document systems are unable to efficiently search technical drawings stored in scanned or image-based formats. By integrating OCR extraction, multimodal reasoning, and CLIP-based visual embeddings, the proposed system enables engineering drawings to be retrieved using natural language descriptions, technical annotations, and visual similarity search. Officers can identify related schematics, component layouts, or visually similar engineering structures without requiring exact drawing numbers or filenames.

The proposed framework can also be applied in maintenance analysis and inspection support activities. Inspection officers and engineering personnel can retrieve historical maintenance records, recurring fault reports, inspection observations, and technical tolerances from multiple repositories simultaneously. This enables faster diagnosis of recurring equipment issues and supports more informed maintenance planning and corrective action analysis. Cross-document reasoning capabilities further allow officers to correlate inspection findings with maintenance procedures and procurement specifications.

Another important application of the system is institutional knowledge preservation. In many technical organizations, operational knowledge regarding document locations, procurement history, and equipment-specific procedures is retained primarily through experienced personnel. When officers are transferred or retired, this knowledge may become inaccessible. The proposed system centralizes technical information into a searchable knowledge repository, reducing dependency on individual expertise and preserving institutional knowledge for future personnel.

The system also has applications in secure defense-oriented AI deployment environments. Since all components operate within a fully on-premise and air-gapped infrastructure, the framework can be used in organizations handling sensitive operational and technical information where cloud-based AI services are not permitted. The architecture can therefore be extended beyond the Indian Coast Guard to other defense organizations, public sector engineering departments, naval maintenance facilities, aerospace support systems, and large-scale industrial enterprises managing sensitive technical documentation.

In addition, the proposed framework provides a foundation for future intelligent enterprise knowledge systems. The scalable architecture can support advanced applications such as multilingual document retrieval, predictive maintenance analytics, AI-assisted technical reporting, automated compliance verification, and role-based personalized retrieval systems. By combining semantic retrieval, multimodal document understanding, grounded language model reasoning, and secure deployment infrastructure, the system demonstrates significant practical applications in modern technical knowledge management and decision support environments.

Chapter 3

3.1 DATASET DESCRIPTION

The proposed system operates on a large and heterogeneous collection of technical and operational documents maintained by the Materiel Directorate of the Indian Coast Guard. The dataset primarily consists of procurement specifications, equipment maintenance manuals, inspection reports, engineering drawings, vendor communications, policy circulars, calibration records, maintenance logs, and Statement of Technical Requirements (SOTR) documents. These records are distributed across multiple storage repositories including departmental drives, local file systems, scanned archives, email attachments, and physical document collections. The absence of a centralized repository makes retrieval and analysis difficult, thereby necessitating the development of an intelligent document indexing and retrieval framework.

The dataset used in the system contains both structured and unstructured information. Structured information includes tables, technical specifications, procurement identifiers, equipment codes, and numerical maintenance records. Unstructured information includes free-text descriptions, inspection observations, technical notes, maintenance procedures, vendor remarks, engineering annotations, and policy instructions. In addition to textual data, the repository also contains multimodal content such as engineering drawings, technical schematics, scanned diagrams, and image-based maintenance documents. These multimodal records contain critical operational information represented visually rather than as machine-readable text.

The document corpus supports multiple file formats including PDF documents, scanned PDFs, Microsoft Word documents (DOC/DOCX), spreadsheets (XLS/XLSX), plain text files, and image formats such as PNG, JPEG, and TIFF. Digital PDF documents generally contain machine-readable text and are processed directly using document parsing techniques. Scanned PDFs and image-based documents require Optical Character Recognition (OCR) processing to extract textual information from scanned pages and engineering annotations. Engineering drawings and schematics are additionally processed through multimodal pipelines to capture both textual and visual information.

During preprocessing, the extracted textual content is cleaned and segmented into smaller contextual chunks to improve semantic retrieval performance. Chunking is necessary because large technical documents often contain multiple independent topics, procedures, and specifications within a single file. Each chunk typically contains a limited number of tokens while preserving contextual continuity through overlapping segmentation techniques. The chunks are then converted into dense vector embeddings using transformer based embedding models. The embeddings represent the semantic meaning of the content and are stored within the Qdrant vector database for efficient similarity-based retrieval.

Along with semantic embeddings, metadata is also associated with every indexed document chunk. The metadata includes document title, file type, source repository, section heading, page number, drawing identifier, procurement category, and timestamp information. This metadata is used during retrieval and citation generation to improve traceability and filtering operations. The system also maintains OCR outputs, extracted annotations, visual embeddings, and semantic descriptions for engineering drawings and image-based technical documents.

For multimodal retrieval, engineering drawings are processed using three independent representations. OCR extraction captures visible annotations, dimensions, labels, and title block details. Visual Question Answering (VQA) models generate semantic descriptions of the schematic content, while CLIP-based visual embeddings enable image similarity search across related engineering diagrams. These representations are indexed separately within the retrieval framework to support text-based, semantic, and visual retrieval mechanisms simultaneously.

The dataset used by the proposed system is continuously expandable, allowing new documents and technical records to be incrementally indexed without requiring complete reprocessing of the existing repository. This ensures scalability and long-term usability as the volume of operational and technical documentation within the organization continues to grow. The combination of textual, visual, structured, and unstructured data makes the dataset highly suitable for Retrieval-Augmented Generation (RAG), semantic retrieval, multimodal reasoning, and automated technical document generation workflows.

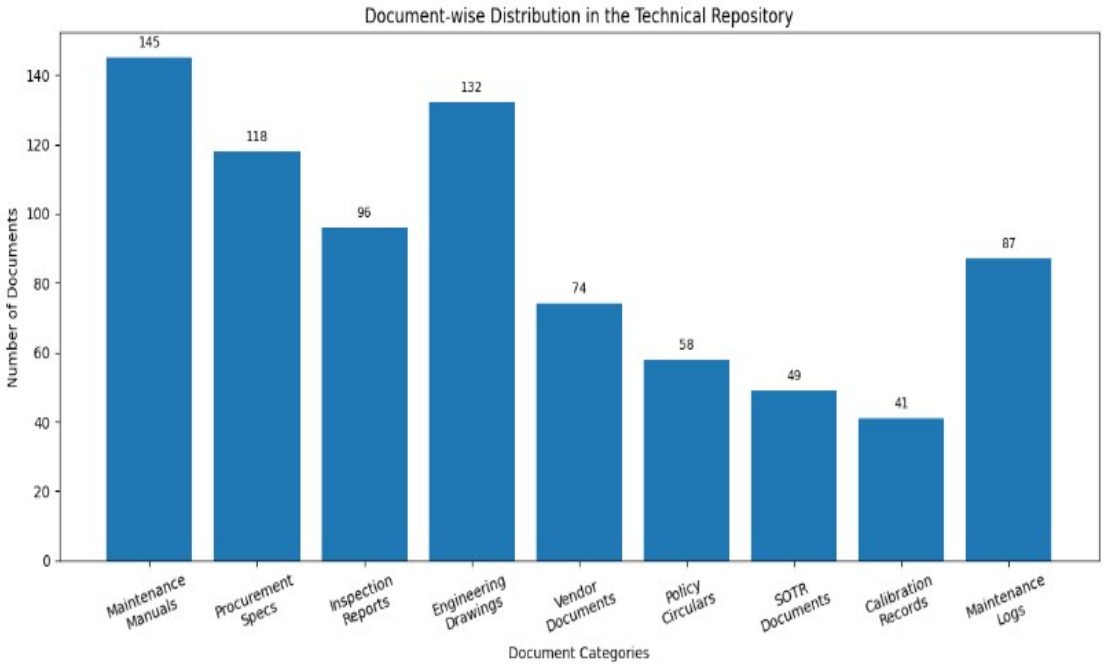


Fig 1: Class-wise Image Distribution of dataset

3.2 METHODOLOGY

The proposed system is developed as an AI-powered Multimodal Knowledge Retrieval and Decision Support platform designed for the Materiel Directorate of the Indian Coast Guard. The system follows a structured architecture that combines document ingestion, semantic retrieval, multimodal understanding, grounded large language model reasoning, and automated document generation within a secure on-premise environment. The complete workflow is organized into multiple stages to ensure efficient processing, indexing, retrieval, and response generation from large collections of technical and operational documents.

Step-1 System Design:

The overall system architecture is divided into multiple interconnected modules consisting of the document ingestion module, preprocessing module, retrieval engine, multimodal processing pipeline, large language model inference module, and user interaction interface. The backend processing module is responsible for handling internal operations such as document parsing, OCR extraction, semantic chunking, vector embedding generation, indexing, retrieval, and answer generation. These components work together to create a unified intelligent retrieval framework capable of handling both textual and visual technical documents.

The interface allows users to search technical repositories, retrieve engineering documents, generate SOTR drafts, and verify citations without requiring database or programming knowledge. The system also supports administrative operations such as document uploading, indexing management, repository updates, and retrieval monitoring. This separation between administrative and operational workflows improves system organization, access control, and deployment management.

Step-2 Document Collection and Repository Organization:

The dataset used in the proposed system consists of heterogeneous technical and operational documents maintained by the Indian Coast Guard Materiel Directorate. These include procurement specifications, equipment maintenance manuals, engineering drawings, inspection reports, calibration records, vendor communications, policy circulars, and Statement of Technical Requirements (SOTR) documents. The repositories contain both structured and unstructured information distributed across multiple storage locations such as departmental drives, scanned archives, email repositories, and local systems.

The system supports multiple document formats including PDF files, scanned PDFs, Word documents, spreadsheets, plain text files, and image-based engineering schematics. Uploaded documents are automatically categorized and organized into predefined repository collections. Metadata such as document title, category, page number, source repository, timestamp, and technical identifiers are extracted and associated with each document during ingestion. This structured organization improves indexing efficiency and enables accurate retrieval operations across large repositories.

Step-3 Document Preprocessing and OCR Extraction:

Document preprocessing plays a critical role in improving retrieval quality and semantic understanding within the system. Digital documents containing machine-readable text are parsed directly using document extraction libraries, while scanned documents undergo Optical Character Recognition (OCR) processing to extract textual content from image-based pages. OCR extraction is especially important for engineering drawings, inspection forms, and scanned maintenance records where critical information is stored visually rather than digitally.

After extraction, the textual content is cleaned and normalized to remove unnecessary formatting inconsistencies, special characters, and duplicate spacing. The processed text is then divided into smaller contextual chunks using overlapping segmentation techniques. Chunking improves semantic retrieval accuracy because large technical documents often contain multiple independent topics within a single file. Overlapping chunk generation preserves contextual continuity between sections and prevents information loss during embedding generation.

Step-4 Embedding Generation and Vector Database Indexing:

Once preprocessing is completed, each document chunk is converted into dense semantic embeddings using transformer-based embedding models. These embeddings represent the contextual meaning of the textual content within high-dimensional vector space.

Along with semantic embeddings, the system also stores metadata associated with every indexed chunk. This metadata includes document titles, section names, page numbers, technical identifiers, source references, and repository categories. Metadata-based filtering improves retrieval precision and supports citation generation during response creation. The vector database architecture allows incremental indexing, enabling new documents to be added to the repository without requiring complete reprocessing of previously indexed data.

Step-5 Multimodal Engineering Drawing Processing:

A major component of the proposed methodology involves multimodal processing of engineering drawings and technical schematics. Since conventional text-based retrieval systems cannot effectively process graphical engineering data, the proposed framework uses a three-path indexing strategy for engineering documents.

The first processing path uses OCR extraction to identify dimensions, annotations, title block information, tolerance values, material specifications, and assembly labels from technical drawings. The second path uses Visual Question Answering (VQA) models to generate semantic descriptions and contextual representations of engineering schematics. The third path employs CLIP-based visual embeddings to support image similarity retrieval across visually related diagrams and schematics. These multiple representations transform engineering drawings into searchable semantic knowledge units capable of supporting textual, contextual, and visual retrieval simultaneously.

Step-6 Hybrid Retrieval and Query Processing:

The retrieval pipeline allows officers to interact with the system using natural language queries without requiring exact filenames or technical identifiers. When a query is submitted, the system first converts the query into semantic embeddings using the same embedding model used during indexing. Multiple retrieval operations are then executed in parallel.

The system performs semantic vector retrieval using similarity search within the vector database. In addition, BM25 keyword retrieval is used to improve exact-match searching for specification IDs, equipment codes, procurement references, and technical terminology. OCR field matching and CLIP-based visual retrieval are also used for engineering drawing-related queries. The outputs from these retrieval mechanisms are combined using Reciprocal Rank Fusion (RRF), which improves retrieval relevance across multiple modalities and document categories.

Step-7 Grounded Response Generation and SOTR Automation:

The retrieved passages, engineering annotations, OCR outputs, metadata, and multimodal descriptions are assembled into a structured context window and supplied to a locally hosted Large Language Model (LLM) for grounded answer generation. The model is configured with strict grounding instructions to ensure that responses are generated only from retrieved evidence rather than unsupported assumptions. Every generated answer includes source citations and document references to improve traceability and factual consistency.

In addition to natural language question answering, the system also supports automated generation of Statement of Technical Requirements (SOTR) documents. Relevant procurement specifications, maintenance procedures, inspection findings, and engineering references are retrieved from indexed repositories and combined into structured draft documents. This automation significantly reduces manual effort and accelerates procurement documentation workflows within the organization.

Step-8 Deployment and System Integration:

The complete system is deployed using Docker-based containerized infrastructure within a fully on-premise and air-gapped environment. All AI models, vector databases, OCR engines, embedding systems, and retrieval components are hosted locally without dependency on external cloud services. The deployment architecture supports secure handling of defense-related technical information while maintaining scalability, modularity, and efficient service orchestration. The integrated framework provides a secure and intelligent knowledge management solution capable of improving operational efficiency, reducing retrieval time, and enabling advanced technical decision support within the Indian Coast Guard Materiel Directorate.

TABLE 1: Comparison between Traditional Search and Proposed RAG Syste

Parameter	Traditional System	Proposed RAG System
Retrieval Method	Exact keyword matching	Semantic and contextual retrieval
Search Capability	Keyword-based	Natural language understanding
Context Awareness	Low	High
Cross-Document Reasoning	Not supported	Supported
Engineering Drawing Retrieval	Limited	Multimodal retrieval
Semantic Understanding	Not available	Transformer embeddings
Citation Support	Not supported	Supported
SOTR Generation	Manual	Automated draft generation
Deployment	Traditional systems	On-premise AI framework
Security	Moderate	Air-gapped deployment

3.2.1 SYSTEM FLOW:

The proposed system follows a structured workflow for intelligent retrieval and processing of technical documents within the Indian Coast Guard Materiel Directorate. Initially, technical documents such as procurement specifications, maintenance manuals, inspection reports, engineering drawings, policy circulars, and SOTR documents are collected from multiple repositories and uploaded into the system. The uploaded documents undergo preprocessing, where textual content is extracted using document parsers and OCR techniques for scanned files and image-based documents. The extracted data is cleaned, segmented into contextual chunks, and converted into semantic vector embeddings using transformer-based embedding models. These embeddings, along with associated metadata, are stored within the Qdrant vector database for efficient semantic retrieval. For engineering drawings and technical schematics, multimodal processing techniques including OCR extraction, Visual Question Answering (VQA), and CLIP-based visual embeddings are applied to enable text-based and visual similarity search. When a user submits a natural language query through the web interface, the system performs hybrid retrieval using semantic vector search, BM25 keyword matching, OCR annotation retrieval, and visual similarity search. Retrieved information from multiple sources is combined and supplied to a locally hosted Large Language Model (LLM) for grounded response generation. The generated response includes relevant citations and references to source documents, ensuring traceability and factual consistency. In addition to query answering, the system also supports automated generation of Statement of Technical Requirements (SOTR) documents by consolidating information from indexed repositories. The complete workflow operates within a secure on-premise and air-gapped infrastructure to ensure protection of sensitive technical and operational data.

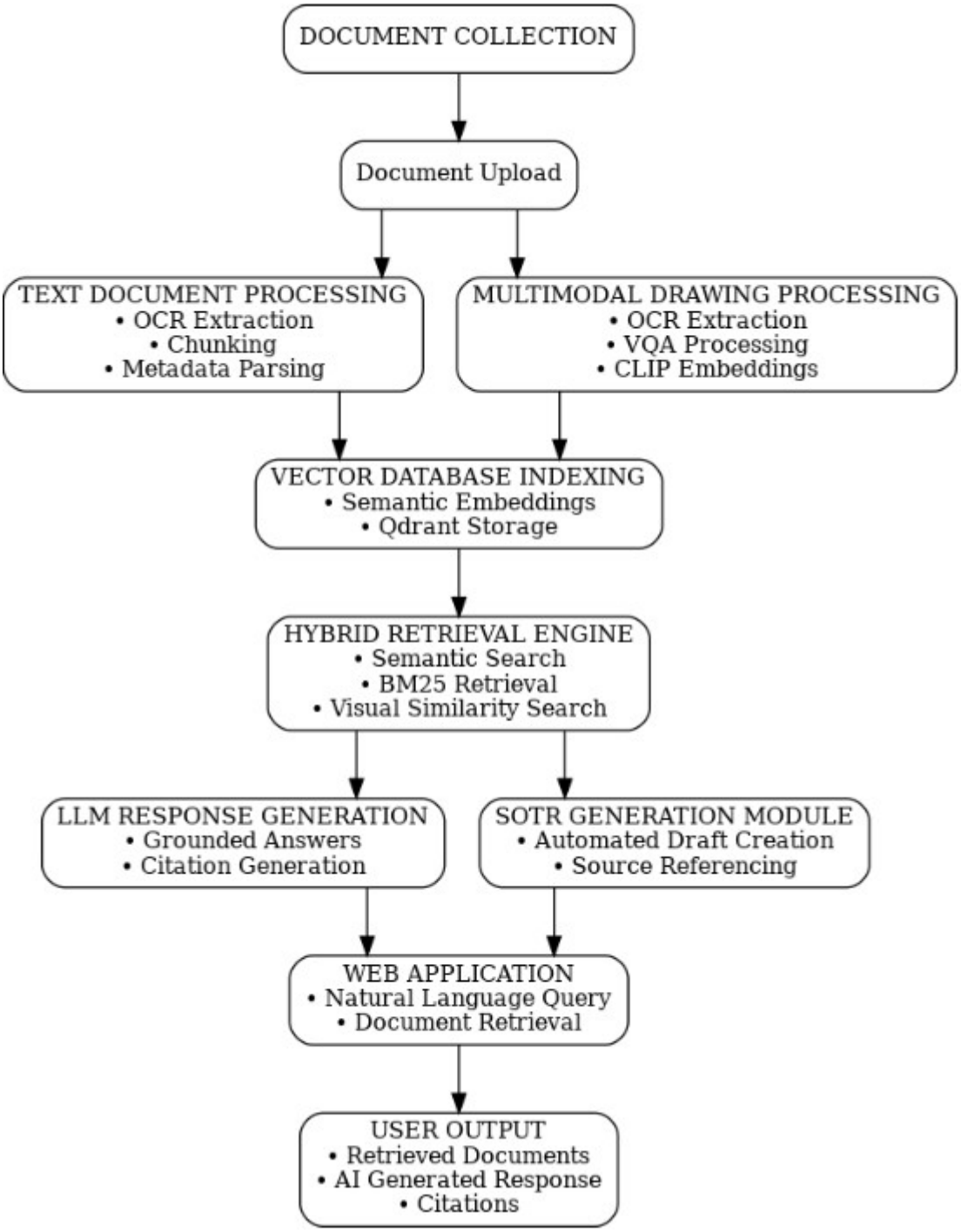


Fig 2: System Architecture

3.2.2 WEB INTEGRATION:

The proposed system is integrated with a web-based application interface to provide a simple and user-friendly platform for interacting with the AI-powered retrieval framework. The web integration layer acts as the communication bridge between the user and the backend processing modules, allowing officers to access technical repositories using natural language queries through a browser-based environment. Users can upload technical documents, search procurement specifications, retrieve engineering drawings, generate SOTR drafts, and view citation-supported responses without requiring technical knowledge of databases or AI systems. The frontend interface is designed to provide secure authentication, query submission, document visualization, and response display functionalities, while the backend handles OCR processing, vector retrieval, multimodal analysis, and grounded response generation using locally hosted AI models. The system supports real-time retrieval and response generation by connecting the web application with the vector database, retrieval engine, and large language model inference pipeline through API-based communication. Since the application is deployed within a fully on-premise and air-gapped infrastructure, all web interactions remain confined within the secure internal network of the Indian Coast Guard, ensuring confidentiality and protection of sensitive technical information.

3.2.2.1 AUTHENTICATION MODULE

The system follows a clear and organized authentication process that includes three main parts: admin login, user registration, and user login. These components are designed to make sure that access to the system is secure while still being easy to use .

ADMIN LOGIN:

The admin login is meant for authorized individuals who are responsible for managing the system. To enter the admin dashboard, valid credentials such as a username and password are required. This helps ensure that only approved users can handle important tasks like uploading datasets, preparing data, training models, and checking results. By limiting access in this way, the system stays protected and avoids any unwanted changes.

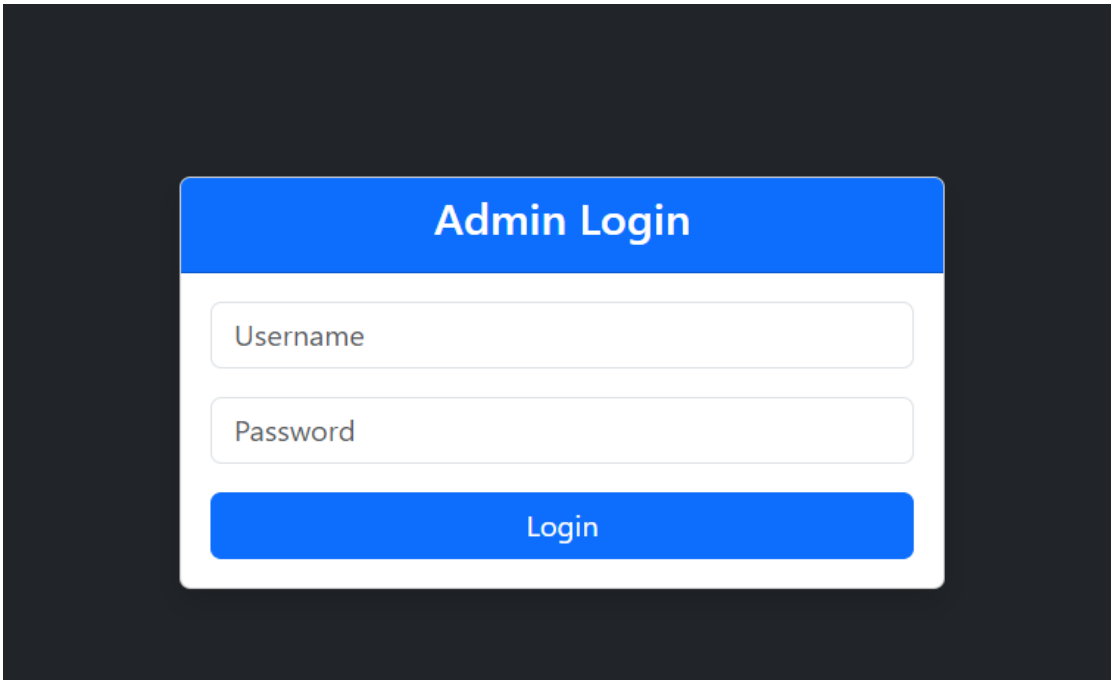


Fig 3: Admin login portal

USER LOGIN:

The user login section is for the approved Indian coast guards who have already registered. By entering their correct username and password, users can access the system and use its features, especially the image prediction part. This step ensures that only genuine users can interact with the system. It also helps in maintaining a smooth and secure experience by keeping user activities organized.

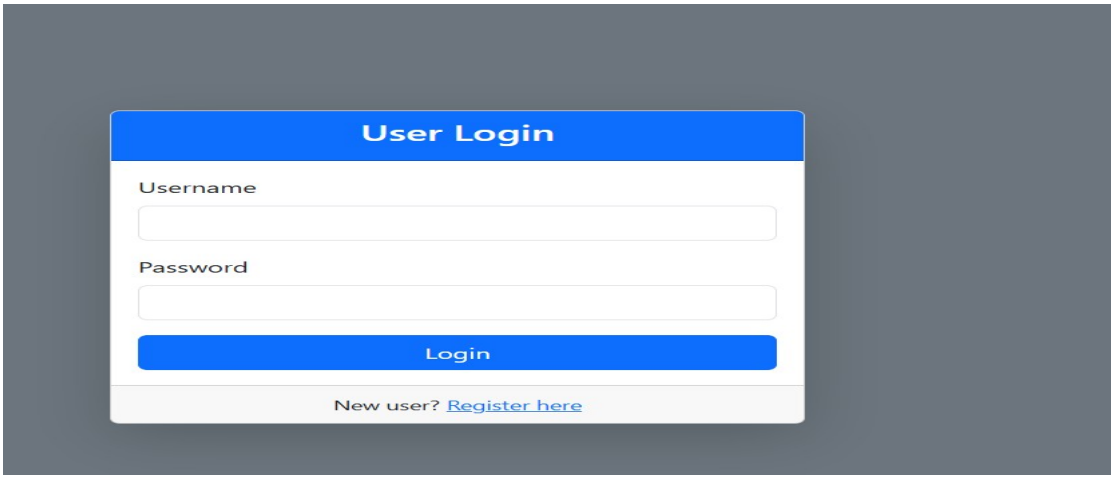


Fig 4: User login portal

3.2.2.2 ADMIN DASHBOARD

The admin dashboard acts as the main control centre of the system, where all important operations are managed in one place. It is designed in a simple and organized way so that the admin can easily perform tasks without confusion. Each feature in the dashboard supports a specific step in the workflow, starting from data handling to final model evaluation. This makes the overall process smooth and easy to manage.

3.3 Dataset Upload:

The dataset upload module is responsible for collecting and organizing technical and operational documents into the system repository for further processing and retrieval. The proposed framework supports multiple file formats including digital PDFs, scanned PDFs, Word documents, spreadsheets, engineering drawings, and image-based technical records. Users or administrators can upload documents through the web-based interface, after which the system automatically validates the uploaded files and categorizes them according to document type and repository structure. During the upload process, metadata such as document title, source category, timestamp, file type, and technical identifiers are extracted and associated with each document to improve indexing and retrieval efficiency. Scanned documents and engineering drawings are additionally routed through OCR and multimodal preprocessing pipelines to extract textual and visual information. The upload module is designed to support incremental repository expansion, allowing new documents to be added continuously without affecting previously indexed data. This automated and organized upload mechanism reduces manual effort, minimizes data management errors, and ensures efficient preparation of documents for semantic indexing and retrieval operations within the system.

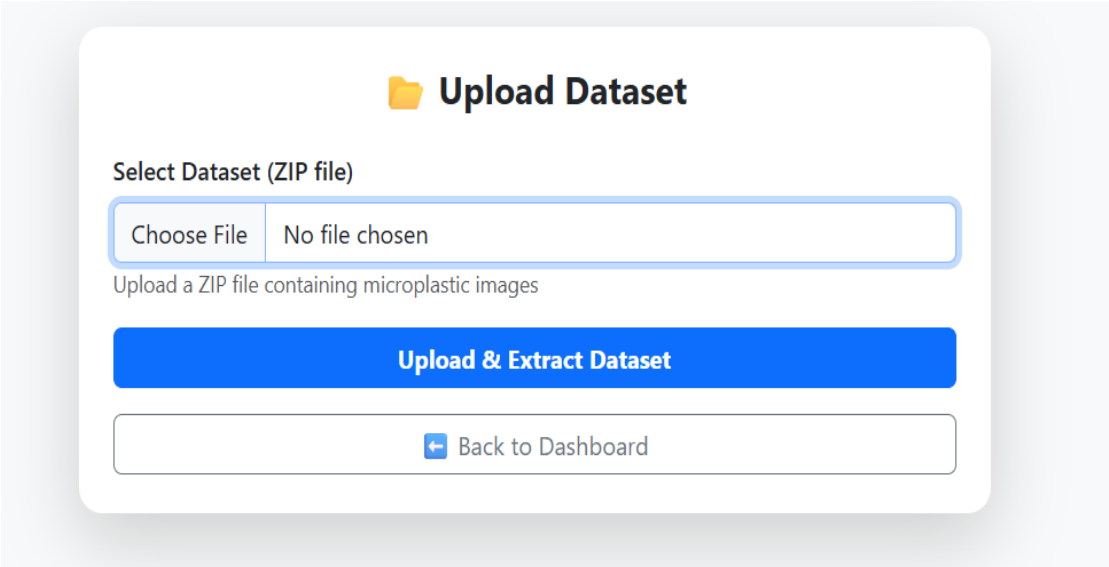


Fig 5: Dataset Upload Section

3.4 DATASET PREPROCESSING:

Dataset preprocessing is an important stage of the proposed system because the uploaded technical repositories contain heterogeneous document formats with varying structures, layouts, and quality levels. The objective of preprocessing is to convert all uploaded documents into a consistent and machine-readable format suitable for semantic retrieval and multimodal analysis. Digital documents containing embedded text are processed directly using document parsing techniques, while scanned PDFs and image-based technical records undergo Optical Character Recognition (OCR) processing to extract textual information.

After text extraction, the system performs cleaning and normalization operations to remove unwanted symbols, formatting inconsistencies, duplicate spaces, and irrelevant characters generated during OCR processing. This improves readability and ensures that the extracted textual content can be indexed efficiently. Since technical documents are often lengthy and contain multiple independent topics, the extracted text is segmented into smaller contextual chunks using overlapping chunking techniques. These chunks preserve semantic continuity while improving retrieval accuracy and reducing computational overhead during embedding generation. In addition to textual preprocessing, the system also performs preprocessing for engineering drawings and multimodal documents. OCR-based annotation extraction, visual feature analysis, and metadata generation are applied to engineering schematics to improve multimodal retrieval performance. Metadata such as document category, page number, repository source, technical references, and engineering identifiers are associated with every processed chunk. Overall, preprocessing ensures consistency, improves semantic understanding, and enhances the efficiency of indexing and retrieval operations within the proposed Retrieval-Augmented Generation framework.

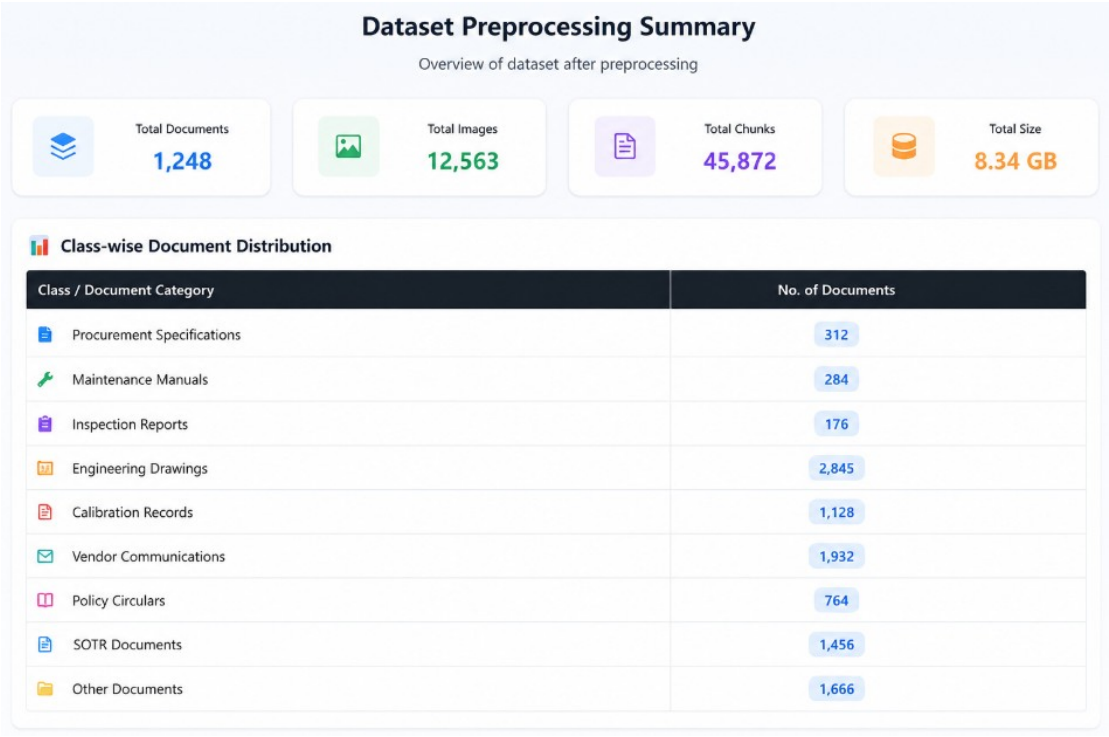


Fig 6: Data Visualisation Module

3.5 OCR AND TEXT EXTRACTION :

OCR and text extraction form one of the core stages of the proposed system because a significant portion of the technical repository consists of scanned documents, engineering drawings, inspection records, and image-based PDFs. Unlike digitally generated documents, scanned records do not contain machine-readable text and therefore cannot be processed directly by semantic retrieval systems. To address this challenge, the system integrates Optical Character Recognition (OCR) techniques to extract textual information from scanned pages and engineering schematics. The OCR pipeline identifies visible textual elements such as technical annotations, dimensions, maintenance notes, specification values, labels, and title block information from uploaded documents.

The extracted textual content is further processed to improve readability and consistency before indexing. Noise generated during OCR processing, including broken characters, duplicated spaces, formatting inconsistencies, and invalid symbols, is removed through cleaning and normalization techniques. For engineering drawings and schematics, OCR extraction plays an especially important role because critical technical information is often embedded within diagrams rather than plain textual reports. The extracted text becomes part of the searchable knowledge repository and is later used during semantic retrieval, citation generation, and grounded response formation.

The OCR extraction module improves the accessibility of archived technical repositories by converting previously non-searchable scanned records into machine-readable content. This significantly reduces manual effort involved in locating technical specifications and engineering references from old maintenance records and scanned procurement documents. By integrating OCR into the retrieval framework, the system ensures that both digital and scanned documents can participate equally in semantic search and multimodal reasoning workflows.

3.6 CHUNKING AND METADATA SEPERATION :

After OCR and document extraction are completed, the textual content undergoes chunking and metadata generation to prepare the repository for semantic indexing and retrieval. Technical documents used within the Materiel Directorate are generally large and contain multiple independent topics, procedures, specifications, and maintenance instructions within a single file. Directly indexing entire documents reduces retrieval accuracy because only certain sections may be relevant to a user query. To solve this problem, the extracted text is divided into smaller contextual segments called chunks.

The chunking process is performed using overlapping segmentation techniques to preserve semantic continuity between adjacent sections. Each chunk contains a manageable amount of contextual information while maintaining logical relationships with neighboring content. This improves semantic retrieval performance because the system retrieves only the most relevant portions of documents instead of entire files. Chunk-based indexing also reduces computational overhead during embedding generation and vector similarity search operations.

Along with chunk generation, metadata is extracted and associated with every indexed segment. Metadata includes information such as document title, page number, section heading, repository category, document type, timestamp, procurement reference number, and engineering identifiers. Metadata improves filtering, citation generation, and traceability during retrieval operations. It also enables users to identify the original source of retrieved information quickly and efficiently. The combination of chunking and metadata generation significantly enhances retrieval precision and improves the overall organization of the technical knowledge repository.

3.7 EMBEDDING GENERATION

Embedding generation is one of the most important components of the proposed Retrieval-Augmented Generation (RAG) framework because it enables semantic understanding of technical documents. After preprocessing and chunking, each document segment is converted into dense vector representations known as embeddings using transformer-based embedding models. As a result, the system can retrieve conceptually related documents even when different technical terminology is used.

The generated embeddings are high-dimensional numerical representations that position semantically similar content closer together within vector space. For example, maintenance instructions, procurement specifications, and inspection findings discussing similar engineering concepts will produce embeddings with similar vector relationships. This semantic representation enables the system to perform contextual retrieval instead of traditional lexical search. Embedding generation therefore improves retrieval relevance and allows officers to search technical repositories using natural language queries.

The embedding generation process also supports multimodal indexing workflows within the system. OCR-extracted annotations, engineering descriptions, and multimodal outputs from technical drawings are converted into embeddings and stored alongside textual document representations. This unified embedding framework allows semantic retrieval to operate across multiple document categories and modalities simultaneously. By integrating transformer-based embeddings into the retrieval pipeline, the system achieves efficient contextual understanding and scalable semantic search capabilities.

3.8 VECTOR DATABASE INDEXING

Once embeddings are generated, they are stored inside the Qdrant vector database for efficient semantic retrieval and similarity search. The vector database acts as the central indexing repository for all processed technical documents, engineering drawings, OCR outputs, metadata, and multimodal representations. Unlike conventional relational databases, vector databases are specifically optimized for handling high-dimensional embeddings and performing nearest-neighbor similarity search operations at large scale.

Each indexed entry within the vector database contains both semantic embeddings and associated metadata. Metadata fields include document category, title, section name,

page number, repository source, engineering identifiers, and timestamps. This allows the retrieval engine to combine semantic similarity search with metadata filtering for improved retrieval precision. The indexing architecture also supports incremental updates, enabling newly uploaded documents to be added without requiring reprocessing of the existing repository.

The vector indexing framework significantly improves retrieval speed and scalability compared to traditional keyword-based document systems. During query processing, the database rapidly identifies embeddings that are semantically closest to the user query and returns the most relevant document chunks. The vector database therefore forms the foundation of the semantic retrieval pipeline and enables efficient contextual search across large and continuously growing technical repositories.

3.9 MULTIMODAL RETRIEVAL

Multimodal retrieval is an important feature of the proposed system because technical repositories within the Indian Coast Guard contain both textual documents and engineering drawings. Conventional retrieval systems are generally limited to text-based search and cannot effectively process graphical engineering information. To overcome this limitation, the proposed framework integrates multimodal retrieval mechanisms capable of handling textual, visual, and OCR-based information simultaneously.

Engineering drawings and technical schematics are processed through multiple independent indexing paths. OCR extraction captures visible annotations such as dimensions, tolerance values, labels, and material specifications from drawings. In addition, Visual Question Answering (VQA) models generate semantic descriptions and contextual understanding of engineering diagrams. CLIP-based visual embeddings are also generated to support image similarity search across visually related schematics and technical layouts.

These multiple representations are indexed together within the retrieval framework, enabling users to retrieve engineering drawings using natural language descriptions, visual similarity, or technical annotations. Multimodal retrieval therefore allows officers to access engineering information more efficiently without requiring exact filenames or drawing identifiers. This capability significantly improves technical verification workflows and enhances the usability of engineering repositories within the organization.

3.10 HYBRID SEARCH

The proposed system uses a hybrid retrieval mechanism that combines semantic vector retrieval with traditional keyword-based search techniques. Although semantic retrieval provides strong contextual understanding, exact keyword matching remains important for technical domains involving specification IDs, equipment codes, engineering references, and procurement identifiers. Hybrid retrieval therefore improves search performance by leveraging the strengths of both retrieval approaches simultaneously.

Semantic retrieval is performed using vector similarity search within the Qdrant vector database, where embeddings related to the user query are identified based on contextual similarity. Alongside semantic retrieval, BM25 keyword retrieval is used to identify documents containing exact technical terms and identifiers. OCR field matching and CLIP-based visual similarity search are additionally used for engineering drawing-related queries.

The outputs from these retrieval mechanisms are combined using Reciprocal Rank Fusion (RRF), which improves overall retrieval relevance across different document types and modalities. Hybrid search significantly improves retrieval precision and reduces the possibility of missing relevant information due to terminology variations. This integrated retrieval strategy enables the system to support accurate, scalable, and context-aware search operations across large technical repositories.

3.11 LLM RESPONSE GENERATION

The final stage of the proposed system involves grounded response generation using locally hosted Large Language Models (LLMs). After retrieval is completed, the relevant document chunks, OCR outputs, engineering annotations, metadata, and multimodal descriptions are assembled into a structured context window. This contextual information is then provided to the language model for answer generation and reasoning.

The language model is configured using strict grounding instructions to ensure that responses are generated only from retrieved evidence rather than unsupported assumptions. Every generated response includes references and citations to the original source documents, allowing officers to verify the authenticity and traceability of retrieved information. This grounded generation approach significantly reduces hallucination and improves factual consistency compared to standalone LLM systems.

In addition to question answering, the LLM module also supports automated generation of Statement of Technical Requirements (SOTR) documents. Relevant procurement specifications, maintenance procedures, engineering references, and inspection findings are synthesized into structured draft documents using retrieved contextual evidence. By combining retrieval with grounded language generation, the proposed system provides an intelligent and reliable decision support framework for technical knowledge management within the Indian Coast Guard.

Chapter 4

Results & Discussion

4.1 Results:

4.1.1 DOCUMENT INGESTION AND PREPROCESSING RESULTS

The document ingestion and preprocessing module was successfully implemented for handling heterogeneous technical and operational records within the Indian Coast Guard Materiel Directorate. The system successfully processed multiple document formats including digital PDFs, scanned PDFs, Word documents, spreadsheets, engineering drawings, and image-based technical records. OCR extraction effectively converted scanned documents and image-based files into machine-readable text, enabling semantic indexing and retrieval of previously non-searchable records. Text cleaning, normalization, and chunking operations improved consistency within the indexed repository and enhanced semantic retrieval performance. Metadata extraction also functioned successfully by associating document titles, section names, page numbers, technical identifiers, and repository categories with indexed document chunks.

4.1.2 EMBEDDING GENERATION AND VECTOR INDEXING RESULTS

The embedding generation and vector indexing module successfully converted processed document chunks into dense semantic vector representations using transformer-based embedding models. These embeddings were indexed within the Qdrant vector database, enabling efficient semantic similarity search across large document repositories. The vector indexing mechanism demonstrated scalable retrieval performance and supported incremental repository expansion without requiring complete reindexing of existing data. Metadata-based filtering further improved retrieval precision by enabling document categorization and source-based filtering during query processing. The vector database architecture significantly improved contextual search capability compared to traditional keyword-only retrieval systems.

4.1.3 MULTIMODAL ENGINEERING DRAWING RETRIEVAL RESULTS

The multimodal retrieval framework successfully processed engineering drawings and technical schematics using OCR extraction, CLIP-based visual embeddings, and Visual Question Answering (VQA) techniques. OCR extraction effectively identified technical annotations, dimensions, title block details, tolerance values, and engineering labels from scanned schematics. CLIP embeddings enabled similarity-based retrieval of visually related engineering diagrams, while VQA-based semantic descriptions improved contextual understanding of engineering layouts. The system allowed users to retrieve engineering drawings through natural language descriptions and visual similarity search instead of depending solely on exact drawing identifiers or filenames. This significantly improved accessibility and usability of engineering repositories.

4.1.4 HYBRID RETRIEVAL AND SEARCH RESULTS

The hybrid retrieval framework combining semantic vector retrieval and BM25 keyword search produced improved retrieval relevance across multiple document categories. Semantic retrieval successfully identified contextually related information even when user queries did not contain exact technical keywords. BM25 keyword retrieval improved search precision for procurement references, specification IDs, equipment codes, and engineering identifiers. Reciprocal Rank Fusion (RRF) effectively combined rankings from multiple retrieval streams and improved the relevance ordering of retrieved results. The integrated retrieval pipeline reduced manual search effort and enabled faster access to technical information distributed across disconnected repositories.

4.1.5 LLM RESPONSE GENERATION RESULTS

The grounded Large Language Model (LLM) response generation module successfully generated context-aware and citation-supported responses using retrieved document evidence. The generated responses demonstrated improved factual consistency because the model was constrained to use retrieved contextual information rather than unsupported assumptions. Citation generation improved traceability and allowed users to verify generated outputs directly against source documents. The system also demonstrated the ability to reject unsupported queries when sufficient evidence was unavailable, thereby reducing hallucination-related issues commonly associated with standalone language models. Natural language interaction through the web interface simplified technical information retrieval for officers without requiring database expertise.

4.1.6 AUTOMATED SOTR GENERATION RESULTS

The automated Statement of Technical Requirements (SOTR) generation module successfully synthesized information from procurement specifications, inspection reports, maintenance manuals, and engineering drawings into structured draft documents. The generated drafts included relevant technical references and citation-supported content extracted from indexed repositories. This significantly reduced the manual effort involved in preparing procurement documentation and improved consistency within technical requirement generation workflows. The automation framework enabled faster preparation of draft SOTR documents while preserving traceability to original technical sources.

4.1.7 SYSTEM DEPLOYMENT AND PERFORMANCE ANALYSIS

The complete system was successfully deployed within a fully on-premise and air-gapped environment using Docker-based containerized infrastructure. All AI models, vector databases, OCR engines, embedding systems, and retrieval components

operated locally without dependency on external cloud services. The deployment architecture demonstrated secure handling of sensitive technical and operational information while maintaining modularity and scalability. Overall system analysis showed considerable improvements in operational efficiency, semantic retrieval accuracy, engineering document accessibility, and institutional knowledge preservation when compared to traditional document retrieval approaches.

4.2 Discussion

4.2.1 Software Details:

The proposed system was developed using a combination of Artificial Intelligence frameworks, retrieval libraries, vector databases, OCR engines, and web development technologiesThe web application interface was developed using Flask to provide a lightweight and efficient communication layer between the user and the backend retrieval modules.

Document parsing and preprocessing operations were implemented using libraries capable of handling PDF documents, Word files, spreadsheets, and image-based technical records. OCR processing was performed using OCR engines for extracting textual information from scanned documents and engineering drawings. Transformer-based embedding models were used for generating semantic vector representations of document chunks, while CLIP-based visual embedding models and Visual Question Answering (VQA) systems were integrated for multimodal engineering drawing analysis.

The semantic embeddings and metadata were indexed using the Qdrant vector database, which enabled efficient similarity-based retrieval across large document repositories. Hybrid retrieval mechanisms combining semantic vector retrieval and BM25 keyword search were implemented to improve retrieval accuracy. Docker containerization technology was used for deployment and orchestration of system components within the secure on-premise environment. The complete software stack was designed to operate without dependency on external cloud services in order to satisfy air-gap security requirements.

4.2.2 Hardware Details

The proposed system was deployed within a local on-premise infrastructure capable of supporting AI inference, semantic retrieval, OCR processing, and multimodal document analysis. A GPU-enabled workstation or server environment was used to support efficient execution of transformer-based embedding models, multimodal AI pipelines, and locally hosted Large Language Models (LLMs). The system required sufficient processing capability for handling OCR extraction, vector embedding generation, retrieval operations, and response generation simultaneously.

The hardware configuration included a multi-core processor for backend processing tasks and retrieval operations. A dedicated GPU with adequate VRAM was required for efficient execution of transformer models, CLIP embeddings, and multimodal

reasoning systems. Sufficient RAM capacity was necessary to support document preprocessing, vector indexing, and concurrent retrieval operations across large repositories. SSD-based storage was used to improve indexing speed and retrieval performance while storing vector databases, embeddings, technical repositories, and AI models locally.

The deployment environment was designed as a fully air-gapped infrastructure without internet dependency to ensure confidentiality of sensitive technical and operational data. All AI models, OCR systems, vector databases, and retrieval components operated entirely within the secure internal network of the Indian Coast Guard. The modular deployment architecture also supports future scalability through additional GPU resources, distributed retrieval infrastructure, and repository expansion.

4.2.3 Limitations:

Although the proposed system demonstrated effective semantic retrieval and multimodal document understanding capabilities, certain limitations were identified during implementation and evaluation. OCR accuracy may reduce when processing low-quality scanned documents, handwritten annotations, damaged engineering drawings, or heavily compressed image-based files. In such cases, incomplete text extraction can affect retrieval quality and citation accuracy.

The performance of semantic retrieval and grounded response generation depends heavily on the quality of embeddings and retrieved contextual information. If relevant document chunks are not retrieved during the retrieval stage, the generated responses may become incomplete or lack sufficient contextual depth. Similarly, multimodal engineering understanding remains dependent on the capabilities of pretrained OCR, CLIP, and VQA models, which may not fully capture highly specialized engineering semantics or domain-specific visual structures.

Another limitation involves computational resource requirements for running locally hosted Large Language Models and multimodal AI pipelines. Since the system operates within a fully on-premise and air-gapped environment, sufficient GPU resources and storage infrastructure are necessary for efficient performance. Large-scale repositories may also increase indexing time and retrieval latency during incremental expansion.

The current implementation primarily supports English-language technical documents and may require additional multilingual embedding and OCR models for supporting multilingual repositories in future deployments. Furthermore, the automated SOTR generation module currently produces draft documents that may still require manual verification and approval by technical officers before operational use. Despite these limitations, the proposed system demonstrates substantial improvements over conventional document retrieval approaches and provides a strong foundation for future enhancements in defense-oriented technical knowledge management systems.

Chapter 5

CONCLUSION & FUTURE WORK

5.1 Conclusion:

The proposed AI-powered Multimodal Knowledge Retrieval and Decision Support System successfully addresses the major challenges associated with technical document management, engineering drawing accessibility, and semantic information retrieval within the Indian Coast Guard Materiel Directorate. The system was designed and implemented using Retrieval-Augmented Generation (RAG) architecture combined with semantic vector retrieval, OCR processing, multimodal engineering understanding, grounded large language model reasoning, and automated SOTR generation. By integrating these technologies into a unified framework, the proposed solution provides an intelligent and scalable platform for handling large collections of technical and operational documents distributed across multiple repositories.

The implementation demonstrated that semantic retrieval using transformer-based embeddings and vector databases significantly improves contextual understanding compared to conventional keyword-based search systems. The hybrid retrieval framework combining semantic vector search and BM25 keyword retrieval improved retrieval relevance for both contextual queries and technical identifiers. The multimodal retrieval pipeline further enhanced engineering drawing accessibility by enabling OCR-based annotation extraction, visual similarity search, and semantic understanding of technical schematics. This allowed users to retrieve engineering information using natural language descriptions rather than relying solely on exact drawing numbers or filenames.

The grounded response generation framework using locally hosted Large Language Models (LLMs) successfully generated context-aware and citation-supported responses while reducing hallucination-related issues through retrieval grounding. Automated Statement of Technical Requirements (SOTR) generation further demonstrated the practical applicability of the system in reducing manual documentation effort and improving workflow efficiency within procurement and engineering operations.

The proposed system also satisfies the security and deployment requirements of defense-oriented environments through fully on-premise and air-gapped deployment architecture. All AI models, vector databases, OCR systems, and retrieval components operate locally without dependency on external cloud services, ensuring protection of sensitive operational and technical information. Overall, the project demonstrates that integrating semantic retrieval, multimodal AI processing, grounded language model reasoning, and secure deployment infrastructure can significantly improve operational efficiency, technical decision-making, and institutional knowledge preservation within large-scale defense document repositories.

5.2 Future Work:

Although the proposed system achieved effective semantic retrieval and multimodal document understanding, several enhancements can be incorporated in future versions to further improve system capabilities and scalability. One important area of future work involves improving OCR and multimodal understanding for highly complex engineering drawings, handwritten annotations, and low-quality scanned documents. Advanced domain-specific OCR models and engineering-aware multimodal reasoning systems can improve extraction accuracy and contextual understanding of technical schematics.

Future improvements may also include integration of advanced reranking and retrieval optimization techniques to improve retrieval relevance and reduce response latency for very large repositories. More sophisticated retrieval architectures incorporating adaptive retrieval, query expansion, and relevance feedback mechanisms can further enhance contextual search performance. Integration of domain-specific fine-tuned embedding models may also improve semantic understanding of specialized defense and engineering terminology.

Another important enhancement involves multilingual support for technical repositories containing documents in multiple languages. Future implementations may incorporate multilingual OCR systems, multilingual embeddings, and language-aware retrieval pipelines to support broader operational environments. Voice-based query interaction and speech-to-text integration can also be added to improve accessibility for field-level personnel and operational officers.

The automated Statement of Technical Requirements (SOTR) generation module can be extended to support more advanced document drafting, automated formatting, template adaptation, and compliance verification workflows. Future systems may also include AI-assisted procurement analysis, predictive maintenance support, anomaly detection within inspection reports, and automated comparison of technical specifications across vendors and procurement cycles.

Scalability improvements can further enable integration with distributed enterprise repositories and real-time synchronization with departmental document management systems. Additional role-based access control mechanisms, audit logging systems, and advanced security policies may also be implemented to support large-scale defense deployments. The proposed framework can further be adapted for use in other defense organizations, naval engineering systems, aerospace maintenance environments, public sector enterprises, and industrial technical knowledge management applications.

Overall, the project establishes a strong foundation for future research and development in AI-driven multimodal retrieval systems, grounded language model applications, and secure enterprise-scale technical knowledge management framework

APPENDICES

```
"""
Citation formatter and validator.
Extracts, validates, and formats source citations from LLM output.
"""

from __future__ import annotations

import re

from loguru import logger

from src.models.schemas import Citation, EvidenceItem, RepresentationType

class CitationFormatter:
    """
    Extract and validate citations from LLM-generated answers.
    Maps [Source N] references to evidence items.
    """

    def extract_citations(
        self, answer: str, evidence_items: list[EvidenceItem]
    ) -> list[Citation]:
        """Extract all [Source N] citations from the answer and map to evidence."""
        cited: list[Citation] = []
        cited_numbers: set[int] = set()

        # Find all [Source N] references
        for match in re.finditer(r"\[Source\s+(\d+)\]", answer, re.IGNORECASE):
```

```
        num = int(match.group(1))
        if num not in cited_numbers and 1 <= num <= len(evidence_items):
            cited_numbers.add(num)
            item = evidence_items[num - 1]
            if item.citation:
                cited.append(item.citation)

    logger.debug(f"Extracted {len(cited)} citations from answer")
    return cited

def validate_citations(
    self, answer: str, citations: list[Citation], evidence_items: list[EvidenceItem]
) -> dict:
    """Validate that all claims in the answer are backed by citations."""
    total_claims = len(re.findall(r"[.!?]", answer))
    cited_count = len(re.findall(r"\[Source\s+\d+\]", answer, re.IGNORECASE))

    return {
        "total_sentences": total_claims,
        "cited_references": cited_count,
        "unique_sources": len(set(c.citation_id for c in citations)),
        "coverage_ratio": min(1.0, cited_count / max(total_claims, 1)),
        "has_unsupported_claims": cited_count < total_claims * 0.5,
    }
```

```
@staticmethod
  def format_citation_footnotes(citations: list[Citation]) -> str:
    """Format citations as footnotes for export."""
    lines = ["\n---", "### Sources"]
    for citation in citations:
      parts = [f"**{citation.citation_id}**:"]
      parts.append(citation.source_file)
      if citation.page_no:
        parts.append(f"Page {citation.page_no}")
      if citation.section:
        parts.append(f"Section: {citation.section}")
      if citation.clause_number:
        parts.append(f"Clause: {citation.clause_number}")
      if citation.drawing_no:
        parts.append(f"Drawing: {citation.drawing_no}")
      if citation.question_tag:
        parts.append(f"({citation.question_tag})")

      lines.append("- " + " | ".join(parts))

    return "\n".join(lines)
```

```
"""
Programmatic compliance checker.
Performs explicit numeric comparisons for tolerance/compliance decisions
before LLM narration. Avoids relying solely on free-form generation.
"""

from __future__ import annotations

import re

from loguru import logger

from src.models.schemas import EvidenceItem

class ComplianceChecker:
    """
    Programmatic compliance checking for numeric thresholds.
    Compares extracted values from drawings against specification requirements.
    """

    def check(self, query: str, evidence_items: list[EvidenceItem]) -> str | None:
        """
        Run programmatic compliance checks on evidence items.
        Returns a formatted string of compliance results, or None.
        """
        # Extract requirement values from spec-type evidence
```

```
        requirements = {}
        actuals = {}

        for item in evidence_items:
            if not item.content:
                continue

            if "clause" in item.label.lower() or "procurement" in item.label.lower():
                # This is a specification — extract requirements
                reqs = self._extract_requirements(item.content)
                for field, req in reqs.items():
                    requirements[field] = {**req, "source": item.source_file, "label": item.label}

            elif any(token in item.label.lower() for token in ["ocr", "vqa", "drawing", "inspection"]):
                # This is a drawing — extract actual values
                actuals_found = self._extract_actuals(item.content)
                for field, val in actuals_found.items():
                    actuals[field] = {**val, "source": item.source_file, "label": item.label}

        if not requirements or not actuals:
            return None
```

```
        # Perform comparisons
        results = []
        for field in requirements:
            if field in actuals:
                req = requirements[field]
                act = actuals[field]
                comparison = self._compare(field, req, act)
                if comparison:
                    results.append(comparison)

        if not results:
            return None

        return "\n".join(results)

    def _extract_requirements(self, text: str) -> dict:
        """Extract threshold requirements from specification text."""
        requirements = {}

        patterns = {
            "radial_clearance": [
                r"(?:bearing\s+)?radial\s+clearance.*?(?:shall|must|be|not exceed).*?(\d+[\.\d]*)\s*(mm|in|µm)?",
            ],
            "tolerance": [
                r"tolerance.*?(?:shall|must|should).*?(?:be|not exceed).*?([±+\-]?\s*[\d.]+\s*(?:mm|in|µm)?)",
                r"(?:±|tolerance)\s*([\d.]+)\s*(mm|in|µm)?",
            ],
            "surface_finish": [
```

```
            r"(?:surface finish|Ra).*?(?:shall|must|should).*?(?:be|not exceed).*?(\d+[\.\d]*)\s*(µm|µm)?",
            r"Ra\s*[≤<]\s*(\d+[\.\d]*)\s*(µm|µm)?",
        ],
        "pressure": [
            r"(?:pressure|design pressure).*?(?:shall|must).*?(\d+[\.\d]*)\s*(psi|bar|kpa|mpa)?",
        ],
        "temperature": [
            r"(?:temperature|design temp).*?(?:shall|must).*?(\d+[\.\d]*)\s*(°[cf]|deg)?",
        ],
    }

    for field, pattern_list in patterns.items():
        for pattern in pattern_list:
            match = re.search(pattern, text, re.IGNORECASE)
            if match:
                value = float(re.search(r"[\d.]+", match.group(1) or match.group(0)).group())
                unit = match.group(2) if match.lastindex and match.lastindex >= 2 else ""
                # Detect comparison type
                comp_type = "max"  # default: value must not exceed
                if "minimum" in text.lower() or "at least" in text.lower():
                    comp_type = "min"
                requirements[field] = {"value": value, "unit": unit or "", "type": comp_type}
                break

    return requirements

def _extract_actuals(self, text: str) -> dict:
    """Extract actual values from drawing/OCR/VQA evidence."""
```

```
        actuals = {}

        patterns = {
            "radial_clearance": [r"(?:bearing\s+)?radial\s+clearance(?:\s*(?:at|=|:))?\s*(\d+[\.\d]*)\s*(mm|in|µm)?"],
            "tolerance": [r"(?:±|tolerance)\s*([\d.]+)\s*(mm|in|µm)?"],
            "surface_finish": [r"Ra\s*(\d+[\.\d]*)\s*(µm|µm)?"],
            "pressure": [r"(\d+[\.\d]*)\s*(psi|bar|kpa|mpa)"],
            "temperature": [r"(\d+[\.\d]*)\s*(°[cf]|deg)"],
        }

        for field, pattern_list in patterns.items():
            for pattern in pattern_list:
                match = re.search(pattern, text, re.IGNORECASE)
                if match:
                    value = float(match.group(1))
                    unit = match.group(2) if match.lastindex >= 2 else ""
                    actuals[field] = {"value": value, "unit": unit or ""}
                    break

        return actuals


    def _compare(self, field: str, requirement: dict, actual: dict) -> str | None:
        """Compare actual vs requirement and return compliance status."""
        req_val = requirement["value"]
        act_val = actual["value"]
        comp_type = requirement.get("type", "max")
        req_unit = requirement.get("unit", "")
        act_unit = actual.get("unit", "")

        if comp_type == "max":
```

```
            if act_val <= req_val:
                status = "✅ COMPLIANT"
                detail = f"{act_val}{act_unit} ≤ {req_val}{req_unit}"
            else:
                status = "❌ NON-COMPLIANT"
                detail = f"{act_val}{act_unit} > {req_val}{req_unit} (exceeds by {act_val - req_val:.4g})"
        elif comp_type == "min":
            if act_val >= req_val:
                status = "✅ COMPLIANT"
                detail = f"{act_val}{act_unit} ≥ {req_val}{req_unit}"
            else:
                status = "❌ NON-COMPLIANT"
                detail = f"{act_val}{act_unit} < {req_val}{req_unit} (below by {req_val - act_val:.4g})"
        else:
            return None

        return (
            f"  {field.upper()}: {status}\n"
            f"    Requirement ({requirement['source']}): {req_val}{req_unit}\n"
            f"    Actual ({actual['source']}): {act_val}{act_unit}\n"
            f"    Result: {detail}"
        )




"""
```

```
Cross-document conflict detector.
Identifies conflicts between values from different sources.
"""

from __future__ import annotations

import re

from loguru import logger

from src.models.schemas import Conflict, EvidenceItem

class ConflictDetector:
    """
    Detects conflicts in LLM output and between evidence items.
    Flags value mismatches across different source documents.
    """

    def detect(self, answer: str, evidence_items: list[EvidenceItem]) -> list[Conflict]:
        """
        Detect conflicts in the answer and between source documents.
        """
        conflicts: list[Conflict] = []

        # 1. Extract conflicts flagged by the LLM itself
        llm_conflicts = self._extract_llm_flagged_conflicts(answer)
        conflicts.extend(llm_conflicts)

        # 2. Detect numeric value conflicts across evidence items
        numeric_conflicts = self._detect_numeric_conflicts(evidence_items)
```

```
        conflicts.extend(numeric_conflicts)

        if conflicts:
            logger.info(f"Detected {len(conflicts)} conflicts")

        return conflicts

    def _extract_llm_flagged_conflicts(self, answer: str) -> list[Conflict]:
        """Extract conflicts explicitly flagged by the LLM."""
        conflicts = []
        patterns = [
            r"⚠\s*CONFLICT:\s*(.+?)(?:\n|$)",
            r"CONFLICT:\s*(.+?)(?:\n|$)",
        ]
        for pattern in patterns:
            for match in re.finditer(pattern, answer, re.IGNORECASE):
                detail = match.group(1).strip()
                conflicts.append(Conflict(
                    field="LLM-detected",
                    source_a="See answer",
                    source_a_value=detail,
                    source_b="See answer",
                    source_b_value=detail,
                    severity="medium",
                ))
        return conflicts

    def _detect_numeric_conflicts(self, evidence_items: list[EvidenceItem]) ->
list[Conflict]:
        """Detect conflicting numeric values across evidence items."""
        conflicts = []
```

```
# Extract all numeric values with field labels from each source
source_values: dict[str, list[tuple[str, str, str]]] = {}
# field → list of (value, source_file, label)

for item in evidence_items:
    if not item.content:
        continue

    nums = self._extract_labeled_numbers(item.content)
    for field, value in nums:
        source_values.setdefault(field, []).append(
            (value, item.source_file, item.label)
        )

# Find conflicts: same field, different values, different sources
for field, values in source_values.items():
    if len(values) < 2:
        continue

    seen = {}
    for value, source, label in values:
        normalized = self._normalize_number(value)
        if normalized is None:
            continue

        for prev_value, (prev_source, prev_label) in seen.items():
            if prev_source != source and abs(normalized - prev_value) > 1e-6:
                conflicts.append(Conflict(
                    field=field,
```

```
                        source_a=f"{prev_source} {prev_label}",
                        source_a_value=str(prev_value),
                        source_b=f"{source} {label}",
                        source_b_value=str(normalized),
                        severity=self._assess_severity(field, prev_value, normalized),
                    ))
            seen[normalized] = (source, label)

        return conflicts

    @staticmethod
    def _extract_labeled_numbers(text: str) -> list[tuple[str, str]]:
        """Extract field-value pairs from text."""
        patterns = [
            (r"(?:tolerance|tol)[:\s]*([±+\-]?\s*[\d.]+\s*(?:mm|in|µm)?)", "tolerance"),
            (r"(?:Ra|surface\s+finish)[:\s]*(\d+[\.\d]*\s*(?:µm|µm)?)", "surface_finish"),
            (r"(?:diameter|dia|ø|⌀)[:\s]*(\d+[\.\d]*\s*(?:mm|in)?)", "diameter"),
                (r"(?:length|width|height|thickness)[:\s]*(\d+[\.\d]*\s*(?:mm|in|cm|m)?)",
"dimension"),
            (r"(?:pressure)[:\s]*(\d+[\.\d]*\s*(?:psi|bar|kpa|mpa)?)", "pressure"),
            (r"(?:temperature|temp)[:\s]*(\d+[\.\d]*\s*(?:°[cf]|deg)?)", "temperature"),
        ]
        results = []
        for pattern, field in patterns:
            for match in re.finditer(pattern, text, re.IGNORECASE):
                results.append((field, match.group(1).strip()))
        return results

    @staticmethod
    def _normalize_number(value_str: str) -> float | None:
        """Extract numeric value from a string."""
```

```
    match = re.search(r"[\d.]+", value_str)
    if match:
        try:
            return float(match.group(0))
        except ValueError:
            pass
    return None

@staticmethod
def _assess_severity(field: str, val_a: float, val_b: float) -> str:
    """Assess conflict severity based on relative difference."""
    if val_a == 0 and val_b == 0:
        return "low"
    relative_diff = abs(val_a - val_b) / max(abs(val_a), abs(val_b), 1e-9)
    if relative_diff > 0.5:
        return "high"
    elif relative_diff > 0.1:
        return "medium"
    return "low"
```

```
"""
Grounded QA engine.
Generates answers based only on retrieved evidence, with citations,
conflict detection, and refusal when evidence is insufficient.
"""

from __future__ import annotations

import re
import time
from pathlib import Path

from loguru import logger

from config.settings import settings
from src.models.schemas import (
    Citation,
    Conflict,
    EvidenceItem,
    QueryMode,
    QueryResponse,
)
from src.reasoning.citation_formatter import CitationFormatter
from src.reasoning.conflict_detector import ConflictDetector
from src.reasoning.compliance_checker import ComplianceChecker
from src.reasoning.llm_client import LLMClient
from src.reasoning.local_reasoner import LocalReasoner
from src.reasoning.refusal_handler import RefusalHandler
```

```
class GroundedQA:
    """
    Grounded question-answering engine.
    Generates answers with inline citations, conflict detection,
    compliance checking, and refusal for insufficient evidence.
    """

    def __init__(self, llm_client: LLMClient):
        self.llm = llm_client
        self.citation_formatter = CitationFormatter()
        self.conflict_detector = ConflictDetector()
        self.compliance_checker = ComplianceChecker()
        self.refusal_handler = RefusalHandler()
        self.local_reasoner = LocalReasoner()
        self._system_prompt = self._load_prompt("qa_system.txt")
        self._user_template = self._load_prompt("qa_user.txt")

    def answer(
        self,
        query: str,
        formatted_context: str,
        evidence_items: list[EvidenceItem],
        mode: QueryMode,
    ) -> QueryResponse:
        """
        Generate a grounded answer to a query.
        """
        start_time = time.time()

        # Check for compliance questions — run programmatic checks first
```

```
compliance_results = None
if self._is_compliance_question(query):
    compliance_results = self.compliance_checker.check(query, evidence_items)

# Check if we have sufficient evidence
refusal = self.refusal_handler.check(query, evidence_items)
if refusal:
    latency = (time.time() - start_time) * 1000
    return QueryResponse(
        answer=refusal,
        citations=[],
        conflicts=[],
        unknown_fields=["Insufficient evidence to answer this question"],
        evidence=evidence_items,
        mode_used=mode,
        latency_ms=latency,
    )

# Build user prompt
user_prompt = self._user_template.format(
    query=query,
    formatted_evidence=formatted_context,
)

# Add compliance context if available
if compliance_results:
    user_prompt += (
        f"\n\nPROGRAMMATIC COMPLIANCE CHECK RESULTS:\n"
        f"{compliance_results}\n"
        f"Incorporate these results into your answer. "
```

```
            f"These numeric comparisons are pre-verified and authoritative."
        )

                    if settings.LOCAL_MODEL_FALLBACK_ENABLED and not self.llm.is_available():
                        raw_answer = self.local_reasoner.answer(query, evidence_items, compliance_results)
        else:
            raw_answer = self.llm.generate(
                system_prompt=self._system_prompt,
                user_prompt=user_prompt,
            )

        # Extract and validate citations
        citations = self.citation_formatter.extract_citations(raw_answer, evidence_items)

        # Detect conflicts between sources
        conflicts = self.conflict_detector.detect(raw_answer, evidence_items)

        # Parse unknown fields
        unknown_fields = self._extract_unknowns(raw_answer)

        # Clean final answer
        answer = self._clean_answer(raw_answer)

        latency = (time.time() - start_time) * 1000

        return QueryResponse(
            answer=answer,
            citations=citations,
            conflicts=conflicts,
```

```
            unknown_fields=unknown_fields,
            evidence=evidence_items,
            mode_used=mode,
            latency_ms=latency,
        )

    def _is_compliance_question(self, query: str) -> bool:
        """Check if the query is about compliance/conformance."""
        compliance_keywords = {
            "comply", "compliance", "compliant", "conform", "conformance",
            "meet", "meets", "satisfy", "satisfies", "within spec",
            "tolerance", "exceed", "violate", "specification requirement",
        }
        query_lower = query.lower()
        return any(kw in query_lower for kw in compliance_keywords)

    def _extract_unknowns(self, answer: str) -> list[str]:
        """Extract UNKNOWN fields from the LLM's response."""
        unknowns = []
        patterns = [
            r"❓ \s*UNKNOWN:\s*(.+?)(?:\n|$)",
            r"UNKNOWN:\s*(.+?)(?:\n|$)",
            r"(?:cannot|could not|unable to)\s+(?:determine|find|verify)\s+(.+?)(?:\.|$)",
        ]
        for pattern in patterns:
            for match in re.finditer(pattern, answer, re.IGNORECASE):
                unknowns.append(match.group(1).strip())
        return unknowns

    def _clean_answer(self, raw: str) -> str:
```

```
    """Clean the raw LLM answer for presentation."""
    # Remove any system-level markers
    raw = re.sub(r"^ANSWER:\s*", "", raw, flags=re.MULTILINE)
    return raw.strip()

@staticmethod
def _load_prompt(filename: str) -> str:
    """Load a prompt template from the config/prompts directory."""
    path = settings.PROMPTS_DIR / filename
    if path.exists():
        return path.read_text(encoding="utf-8")
    logger.warning(f"Prompt template not found: {path}")
    return ""
```

```
"""
LLM inference client for the reasoning engine.
Connects to a local OpenAI-compatible backend.
"""

from __future__ import annotations

from pathlib import Path

from loguru import logger
from openai import OpenAI

from config.settings import settings

class LLMClient:
    """
    Client for local LLM inference via OpenAI-compatible API.
    Supports vLLM and Ollama backends.
    """

    def __init__(
        self,
        base_url: str = settings.REASONING_BASE_URL,
        model_name: str = settings.REASONING_MODEL_NAME,
        model_path: Path = settings.REASONING_MODEL_PATH,
        api_key: str = settings.REASONING_API_KEY,
        max_tokens: int = settings.REASONING_MAX_TOKENS,
        temperature: float = settings.REASONING_TEMPERATURE,
    ):
        self.client = OpenAI(base_url=base_url, api_key=api_key)
```

```
    self.model_name = model_name
    self.model_path = Path(model_path)
    self.max_tokens = max_tokens
    self.temperature = temperature

def generate(
    self,
    system_prompt: str,
    user_prompt: str,
    max_tokens: int | None = None,
    temperature: float | None = None,
) -> str:
    """Generate a completion from the LLM."""
    try:
        response = self.client.chat.completions.create(
            model=self.model_name,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_prompt},
            ],
            max_tokens=max_tokens or self.max_tokens,
            temperature=temperature or self.temperature,
        )
        return response.choices[0].message.content.strip()
    except Exception as exc:
        logger.error(f"LLM generation failed: {exc}")
        raise

def generate_with_history(
    self,
```

```
    messages: list[dict[str, str]],
    max_tokens: int | None = None,
    temperature: float | None = None,
) -> str:
    """Generate a completion with full message history."""
    try:
        response = self.client.chat.completions.create(
            model=self.model_name,
            messages=messages,
            max_tokens=max_tokens or self.max_tokens,
            temperature=temperature or self.temperature,
        )
        return response.choices[0].message.content.strip()
    except Exception as exc:
        logger.error(f"LLM generation failed: {exc}")
        raise

def is_available(self) -> bool:
    """Check if the local LLM backend is reachable and local assets exist."""
    if not self.model_path.exists():
        return False
    try:
        models = self.client.models.list()
        return len(models.data) > 0
    except Exception:
        return False
```

```
"""
Deterministic grounded answer and SOTR synthesis for local/offline runs.
"""

from __future__ import annotations

import re

from src.ingestion.ocr_postprocessor import OCRPostprocessor
from src.models.schemas import EvidenceItem
from src.offline_support import split_sentences, tokenize, unique_preserve_order

STOPWORDS = {
    "the", "a", "an", "is", "are", "was", "were", "of", "for", "to", "and", "or",
    "what", "which", "with", "does", "do", "in", "on", "at", "this", "that", "it",
    "all", "any", "from", "into", "their", "there", "show", "list",
}

class LocalReasoner:
    def __init__(self) -> None:
        self.postprocessor = OCRPostprocessor()

    def answer(self, query: str, evidence_items: list[EvidenceItem], compliance_results:
str | None = None) -> str:
        if compliance_results:
            return self._answer_compliance(evidence_items, compliance_results)

        ranked = self._ranked_sentences(query, evidence_items)
        if not ranked:
            return "UNKNOWN: Insufficient grounded evidence to answer this query."
```

```
        answer_sentences = []
        for sentence, item in ranked[:4]:
            answer_sentences.append(self._with_citation(sentence, item))
        return " ".join(unique_preserve_order(answer_sentences))

    def generate_section(
        self,
        section_num: int,
        section_title: str,
        equipment_name: str,
        evidence_items: list[EvidenceItem],
        compliance_results: str | None = None,
    ) -> str:
        builders = {
            1: self._section_identification,
            2: self._section_specifications,
            3: self._section_performance,
            4: self._section_inspection,
            5: lambda items: self._section_compliance(items, compliance_results),
            6: self._section_maintenance,
            7: self._section_procurement,
            8: lambda items: self._section_recommendations(items, compliance_results),
        }
        content = builders.get(section_num, self._section_generic)(evidence_items)
        if not content.strip():
            return (
                f"UNKNOWN: Section {section_num} ({section_title}) could not be populated from the available evidence. "
                "[Source 1]"
            )
        return content
```

```
    def _section_identification(self, evidence_items: list[EvidenceItem]) -> str:
        item = self._first_item(evidence_items, include=("VQA-Q7", "OCR"))
        if not item:
            return ""
        content = item.content.split("A:", 1)[-1].strip()
        content = content.replace("\n", " ")
        return self._with_citation(content, item)


    def _section_specifications(self, evidence_items: list[EvidenceItem]) -> str:
        drawing_item = self._first_item(evidence_items, include=("OCR",))
        procurement_item = self._first_item(evidence_items, include=("Procurement Clause",))
        sentences = []
        if drawing_item:
            raw = drawing_item.content
            dimensions = re.findall(r"Ø\d+mm|L=\d+mm|H=\d+mm", raw)
            if dimensions:
                sentences.append(self._with_citation(
                    f"The drawing records the key dimensions as {', '.join(unique_preserve_order(dimensions))}",
                    drawing_item,
                ))
            tolerance_bits = []
            for pattern in [r"0\.02-0\.05 mm", r"±0\.05mm", r"Ra 1\.6µm", r"Ra 3\.2µm"]:
                match = re.search(pattern, raw, re.IGNORECASE)
                if match:
                    tolerance_bits.append(match.group(0))
            if tolerance_bits:
                sentences.append(self._with_citation(
```

```
                f"Drawing tolerances and finish notes include {', '.join(unique_preserve_order(tolerance_bits))}",
                drawing_item,
            ))
        material_match = re.search(
            r"Materials:\s*ASTM A105 casing,\s*316SS impeller and shaft",
            raw,
            re.IGNORECASE,
        )
        if material_match:
            sentences.append(self._with_citation(
                "Material callouts on the drawing identify ASTM A105 for the casing and 316SS for the impeller and shaft",
                drawing_item,
            ))
    if procurement_item:
        sentences.append(self._with_citation(
            "Procurement requirements reinforce the cited material and tolerance limits for the assembly",
            procurement_item,
        ))
    return " ".join(sentences)


def _section_performance(self, evidence_items: list[EvidenceItem]) -> str:
    return self._compose_keyword_section(
        evidence_items,
        keywords={"flow", "pressure", "temperature", "duty", "capacity", "operating"},
        limit=3,
    )


def _section_inspection(self, evidence_items: list[EvidenceItem]) -> str:
```

```
        return self._compose_keyword_section(
            evidence_items,
            keywords={"inspection", "finding", "measured", "clearance", "condition", "reported", "date"},
            limit=3,
        )


    def _section_compliance(self, evidence_items: list[EvidenceItem], compliance_results: str | None) -> str:
        if not compliance_results:
            item = self._first_item(evidence_items)
            return self._with_citation("UNKNOWN: Compliance could not be verified from the retrieved evidence", item) if item else ""

        lines = []
        status = "UNKNOWN"
        if "NON-COMPLIANT" in compliance_results:
            status = "NON-COMPLIANT"
        elif "COMPLIANT" in compliance_results:
            status = "COMPLIANT"

        lines.append(f"{status}: programmatic comparison was completed across the retrieved drawing, specification, and inspection evidence.")
        for raw_line in compliance_results.splitlines():
            cleaned = raw_line.strip().lstrip("-")
            if not cleaned:
                continue
            if cleaned.startswith(("Requirement", "Actual", "Result")) or cleaned[0].isalnum():
                lines.append(cleaned.rstrip(".") + ".")

        cited = []
```

```
        for item in evidence_items[:3]:
            cited.append(self._citation(item))
        return " ".join(
            f"{line.rstrip('.')} {' '.join(cited)}."
            for line in lines[:3]
        )

    def _section_maintenance(self, evidence_items: list[EvidenceItem]) -> str:
        return self._compose_keyword_section(
            evidence_items,
                keywords={"maintenance", "interval", "inspect", "lubricate", "replace",
"consumable", "procedure"},
            limit=4,
        )

    def _section_procurement(self, evidence_items: list[EvidenceItem]) -> str:
        return self._compose_keyword_section(
            evidence_items,
              keywords={"vendor", "delivery", "inspection", "certificate", "procurement",
"clause", "shall"},
            limit=4,
        )

            def _section_recommendations(self, evidence_items: list[EvidenceItem],
compliance_results: str | None) -> str:
                inspection_item = self._first_item(evidence_items, include=("Inspection
Finding",))
            procurement_item = self._first_item(evidence_items, include=("Procurement
Clause",))
        sentences = []
            if compliance_results and "NON-COMPLIANT" in compliance_results and
inspection_item and procurement_item:
```

```
            sentences.append(
                f"Recommendation: replace the bearing assembly and restore the measured clearance to the specified limit before release to service "
                f"{self._citation(inspection_item)} {self._citation(procurement_item)}."
            )
        if inspection_item:
            sentences.append(self._with_citation(
                "Recommendation: repeat inspection after corrective work and capture the updated measurements in the next inspection report",
                inspection_item,
            ))
        return " ".join(sentences)


    def _section_generic(self, evidence_items: list[EvidenceItem]) -> str:
        ranked = self._ranked_sentences("", evidence_items)
        if not ranked:
            return ""
        return " ".join(
            self._with_citation(sentence, item)
            for sentence, item in ranked[:3]
        )


                def _answer_compliance(self, evidence_items: list[EvidenceItem], compliance_results: str) -> str:
        drawing_item = self._first_item(evidence_items, include=("OCR",))
            procurement_item = self._first_item(evidence_items, include=("Procurement Clause",))
                inspection_item = self._first_item(evidence_items, include=("Inspection Finding",))


        sentences = []
        if drawing_item:
```

```
        structured = self.postprocessor.extract_fields(drawing_item.content)
        tolerance_bits = unique_preserve_order(structured.tolerances)[:2]
        if tolerance_bits:
            sentences.append(self._with_citation(
                f"Drawing {structured.drawing_no or drawing_item.source_file} specifies {', '.join(tolerance_bits)}",
                drawing_item,
            ))
    if procurement_item:
        sentences.append(self._with_citation(
            "Procurement requirements set the governing tolerance and acceptance limits for the assembly",
            procurement_item,
        ))
    if inspection_item:
        sentences.append(self._with_citation(
                "The latest inspection reports the measured condition recorded in the inspection evidence",
            inspection_item,
        ))


    if "NON-COMPLIANT" in compliance_results:
            sentences.append("Result: NON-COMPLIANT because the inspection measurement exceeds the specified limit.")
    elif "COMPLIANT" in compliance_results:
            sentences.append("Result: COMPLIANT because the measured condition remains within the cited limit.")
    else:
            sentences.append("Result: UNKNOWN because the retrieved evidence is insufficient for a deterministic compliance call.")


    if procurement_item:
```

```
            sentences[-1] = self._with_citation(sentences[-1], procurement_item)
        elif drawing_item:
            sentences[-1] = self._with_citation(sentences[-1], drawing_item)

        return " ".join(sentences)

    def _compose_keyword_section(
        self,
        evidence_items: list[EvidenceItem],
        keywords: set[str],
        limit: int,
    ) -> str:
        ranked = []
        for item in evidence_items:
            for sentence in split_sentences(item.content):
                score = self._score_sentence(sentence, keywords)
                if score > 0:
                    ranked.append((score, sentence, item))
        ranked.sort(key=lambda row: row[0], reverse=True)

        sentences = []
        for _, sentence, item in ranked[:limit]:
            sentences.append(self._with_citation(sentence, item))
        return " ".join(unique_preserve_order(sentences))

    def _ranked_sentences(self, query: str, evidence_items: list[EvidenceItem]) -> list[tuple[str, EvidenceItem]]:
        query_terms = {token for token in tokenize(query) if token not in STOPWORDS}
        ranked = []
        for item in evidence_items:
            for sentence in split_sentences(item.content):
```

```
            score = self._score_sentence(sentence, query_terms)
            if score <= 0 and query_terms:
                continue
            ranked.append((score, sentence, item))
        ranked.sort(key=lambda row: row[0], reverse=True)
        return [(sentence, item) for _, sentence, item in ranked]

    @staticmethod
    def _score_sentence(sentence: str, query_terms: set[str]) -> int:
        if not query_terms:
            return len(tokenize(sentence))
        sentence_terms = set(tokenize(sentence))
        return len(sentence_terms.intersection(query_terms))

    @staticmethod
    def _first_item(evidence_items: list[EvidenceItem], include: tuple[str, ...] | None =
None) -> EvidenceItem | None:
        for item in evidence_items:
            if not include or any(token in item.label for token in include):
                return item
        return evidence_items[0] if evidence_items else None

    @staticmethod
    def _citation(item: EvidenceItem | None) -> str:
        if not item or not item.citation:
            return "[Source 1]"
        return f"[{item.citation.citation_id}]"

    def _with_citation(self, text: str, item: EvidenceItem | None) -> str:
        cleaned = text.strip()
        if cleaned.endswith("."):
```

```
            cleaned = cleaned[:-1].rstrip()
        return f"{cleaned} {self._citation(item)}."
```

"""

Refusal / UNKNOWN handler.

Determines when evidence is insufficient and generates appropriate refusal responses.

```
"""

from __future__ import annotations

from loguru import logger

from src.models.schemas import EvidenceItem

class RefusalHandler:
    """
    Determines when evidence is insufficient to answer a query
    and provides structured UNKNOWN responses instead of hallucinating.
    """

    # Minimum thresholds for answering
    MIN_EVIDENCE_ITEMS: int = 1
    MIN_RELEVANCE_SCORE: float = 0.01
    MIN_CONTENT_LENGTH: int = 50  # characters

    def check(self, query: str, evidence_items: list[EvidenceItem]) -> str | None:
        """
        Check if evidence is sufficient to answer the query.
        Returns None if evidence is adequate, or a refusal message if not.
        """
        # No evidence at all
        if not evidence_items:
            logger.info("Refusal: No evidence items retrieved")
            return (
                "❓ UNKNOWN: No relevant evidence was found in the document collection
"
                "to answer this question. Please verify that the referenced documents "
```

```
            "have been ingested, or rephrase the query."
        )

    # Filter to meaningful evidence
    meaningful = [
        item for item in evidence_items
        if item.content and len(item.content) >= self.MIN_CONTENT_LENGTH
    ]

    if len(meaningful) < self.MIN_EVIDENCE_ITEMS:
        logger.info("Refusal: Insufficient meaningful evidence")
        return (
            "❓ UNKNOWN: The retrieved evidence does not contain sufficient "
            "information to answer this question reliably. "
            f"Found {len(evidence_items)} items but none with adequate content."
        )

    # Check relevance scores
    max_score = max(item.score for item in evidence_items)
    if max_score < self.MIN_RELEVANCE_SCORE:
        logger.info(f"Refusal: Best relevance score {max_score:.3f} below threshold")
        return (
            f"❓ UNKNOWN: The retrieved evidence has low relevance "
            f"(best match score: {max_score:.2f}). The documents may not contain "
            f"information directly addressing this question."
        )

    return None  # Evidence is sufficient
```

REFERENCES

[1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459–9474

[2] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. arXiv Preprint arXiv:2103.00020.

[3] Sharma, C. (2025). Retrieval-augmented generation: A comprehensive survey of architectures, enhancements, and robustness frontiers. arXiv Preprint arXiv:2506.00054.

[4] Pan, J. J., Wang, Y., & Madden, S. (2023). Survey of vector database management systems. arXiv Preprint arXiv:2310.14021.

[5] Li, Z. (2025). Retrieval-augmented generation for educational application. Smart Learning Environments, 12(1), 1–18.

[6] Khan, A. A., Hasan, M. T., Kemell, K. K., Rasku, J., & Abrahamsson, P. (2024). Developing retrieval augmented generation (RAG) based LLM systems from PDFs: An experience report. arXiv Preprint arXiv:2410.15944.

[7] Ockerman, S., Gueroudji, A., Oh, S. Y., Underwood, R., Chia, N., Chard, K., Ross, R., & Venkataraman, S. (2025). Exploring distributed vector databases performance on HPC platforms: A study with Qdrant. ACM Digital Library.

[8] Picha, K. (2024). A retrieval-augmented generation based large language model benchmarked on a novel dataset. ResearchGate.

[9] Zhao, P. (2026). Retrieval-augmented generation for AI-generated content. Frontiers of Computer Science, 20(2), 1–15.

[10] OpenAI. (2021). CLIP: Connecting text and images. OpenAI.

[11] Qdrant. (2025). Qdrant documentation. Qdrant Technologies.

[12] Qdrant. (2025). Qdrant vector search engine. Qdrant Technologies.

[13] Qdrant. (2025). The hitchhiker's guide to vector search. Qdrant Technologies.

[14] OpenAI. (2021). CLIP (Contrastive Language–Image Pre-Training). GitHub Repository.

[15] Chen, X. (2025). A research of challenges and solutions in retrieval augmented generation systems. Highlights in Science, Engineering and Technology, 128, 102–110.